SYMPHONY v2.0 Search Anything...

Overview

Introduction

XTS API is a high-performance trading API designed to deliver seamless integration with advanced trading platforms. It enables developers, brokers, and third-party vendors to build reliable, low-latency trading and investment solutions with ease.

The API offers REST-like endpoints for executing and modifying orders in real time, managing portfolios, accessing live market data, and implementing complex trading strategies. All communication is secured and handled via JSON-formatted requests and responses, with standard HTTP codes used to indicate success or failure states.

Users receive a unique API key and secret during application registration, along with the option to register a redirect URL for secure authentication flows. The API is optimized for high throughput, scalable performance, and supports large concurrent user bases, ensuring stability even in volatile market conditions.

Disclaimer: The API access and data are strictly for registered clients and vendors. Users should safeguard their trading credentials and never embed sensitive secrets directly in client applications.

Note : API key has to be created by client itself not anyone else. Please do not share trading id / Password with anyone.

API Environment URLs

POST `https://developers.symphonyfintech.in/hostlookup` Copy

Libraries and SDKs

Below is a list of pre-built client libraries and SDKs for Symphony API written in various programming languages that can be used to interact with the APIs without having to make raw HTTP calls

Symphony Python Library

Python API SDK

Symphony Node JS Library

NodeJs Interactive API

NodeJs MarketData API

Symphony Java Library

Java API SDK

SYMPHONY v2.0

to Get Access to XTS Trimmed Interactive API Services

To access **Trimmed Interactive API services,** you need a public key. Follow these steps:

1. **Register on the Broker's API Dashboard**

   - Visit your broker's API dashboard page.

   - Register yourself as an API user.

2. **Verify Your Email**

   - After registration, you'll receive a verification link via email.

   - Click the link to validate your email ID.

   - You will be redirected to the login page.

3. **Subscribe to the API Service**

   - Log in to the API dashboard: This is sample url https://developers.symphonyfintech.in/dashboard

   - Select **Trimmed Interactive API subscription** and provide the required details.

   - Wait for an approval email from the broker's support team.

4. **Receive Your API Keys**

   - Once approved, you'll get an email containing your **appKey** and **secretKey**.

   - Keep these keys safe for future use.

5. **Important Notes**

   - Only **One session** is allowed per **appKey-secretKey** combination.

   - Do **not share** your keys with others.

   - If you suspect your keys are compromised, log in to the API dashboard and **regenerate** new keys then update them in your source code or configuration.

How To Access XTS Trimmed Interactive API Services

Register > Verify Email > Subscrive > Approval > Get Key

Authentication Token Requirement

For all API requests **the login API**, an **authentication token** must be included in the request header.

- The token is generated only after a **successful user login** .
- It remains valid for **24 hours** from the time of generation. .

SYMPHONY v2.0

```
Headers:
Content-Type: application/json
authorization: eyJ1c2VySUQiOiJSVVBFU0giLCJpYXQiOjE1NTIxMjIwOTEsImV4cCI6MTU1MjIwODQ5MX0
```

Copy Show Full

Note: Default values are examples provided in response.

Request & Response Structure

All GET and DELETE request parameters go as query parameters, and POST and PUT parameters as form-encoded. The user has to input an authorization in the header for every request. All request parameters as application/json parameters. Responses from the API will be in JSON

Successful request:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "type": "success",
  "code": "status code",
  "description": "description message",
  "result":
}
```

Copy Show Full

All responses from the API server are JSON with the content-type application/json unless explicitly stated otherwise. A successful 200 OK response always has a JSON response body with a status key with the value success. The result key contains the full response payload.

Failed request:

```
HTTP/1.1 400 Server error
Content-Type: application/json
```

```
{
  "type": "error",
  "code": "error code",
```

SYMPHONY v2.0

Copy Show Full

Error Handling

Common HTTP error codes

Code	Error Description
400	Missing or bad request parameters or values
404	Request resource was not found
429	Too many requests to the API (rate limiting)
500	Something unexpected went wrong

Rate Limit

Here is the rate limit for Interactive API calls

Why Throttling is Important

Throttling ensures that API requests are distributed evenly over time to:

- Prevent server overload and downtime during peak hours
- Maintain fair access to resources for all clients
- Protect the stability and performance of the trading system during volatile market conditions
- Avoid accidental denial-of-service caused by excessive automated requests

Rate Limits Table

API Name	Endpoint	Method	Limit (/
Unified Order API Throttle Limit	Unified Order API Throttle Limit	Note	10
Balance	/user/balance	GET	1
Profile	/user/profile	GET	1
Brokerage Calculator	/user/calculatebrokerage	POST	10
Cancel All	/orders/cancelall	POST	1

Tradebook	/orders/trades	GET	1
Spread Orderbook	/orders/spread	GET	1
Place GTT Order	/orders/gttorder	POST	1
Modify GTT Order	/orders/gttorder	PUT	1
Cancel GTT Order	/orders/gttorder	DELETE	1
GTT Orderbook	/orders/gttorderbook	GET	1
Holding	/portfolio/holdings	GET	1
Position	/portfolio/positions	GET	1
Position Convert	/portfolio/positions/convert	PUT	10
Regular Order Margin	/orders/margindetails	POST	10
Regular Order Modify Margin	/orders/modifyordermargindetails	POST	10
Cover Order Margin	/orders/comargindetails	POST	10
Cover Order Modify Margin	/orders/comodifymargindetails	POST	10
Bracket Order Margin	/orders/bomargindetails	POST	10
Exchange Status	/status/exchange	GET	1
Exchange Message	/messages/exchange	GET	1

Note : **Unified Order Throttle Limit:**

This is a combined rate limit applied across all order-related APIs listed below.
The standard limit is 10 requests per second, shared cumulatively among these APIs (Place Order,Modify Order,Cancel Order,Place BO (Bracket Order),Modify BO,Cancel BO,Place CO (Cover Order),Exit Cover,Place Spread Order,Modify Spread Order,Cancel Spread Order).

Enums with Numeric Constants

Each Enum now includes its corresponding numeric constant for easier mapping in client applications.

1. ExchangeSegments

Enum Name	Numeric Code	Description
`NSECM`	1	The equity cash segment of NSE where stocks of listed companies are traded for delivery

SYMPHONY v2.0

NSECD	3	Currency trading segment at NSE for exchange-traded currency futures and options (USI
BSECM	11	Cash/equity segment of BSE, the oldest stock exchange in Asia, where stocks are bought
BSEFO	12	Derivatives segment of BSE offering equity derivatives, index derivatives, and stock future
BSECD	13	Currency derivatives platform of BSE that allows trading in exchange-listed currency futu
MCXFO	51	Derivatives trading segment of MCX for commodities such as gold, silver, crude oil, natura products.
NCDEX	21	A leading exchange focused on agricultural commodity futures like soybean, chana, guar,

2. ProductType

Enum Name	Numeric Code	Description
MIS	1	Intraday trading product type where positions must be squared off before market close. P compared to delivery.
NRML	2	Standard product type used for carrying forward positions overnight in derivatives or cash intraday square-off.
CNC	4	Delivery-based product type for equity trades. Shares are delivered to the Demat account
CO	8	Intraday order type where a market or limit order is placed along with a mandatory stop-lo leverage while controlling risk.
BO	16	Advanced order type where a main order is placed with both a target (profit booking) and executed, the other is automatically cancelled. Can also include a trailing stop-loss.
MTF	32	Allows investors to buy stocks by partly paying upfront and borrowing the balance from th carried forward as long as margin requirements are met.

3. OrderType

Enum Name	Numeric Code	Description
Market	1	An order to buy or sell immediately at the best available price in the market.
Limit	2	An order to buy or sell at a specified price (or better).
StopMarket	4	An order that becomes a market order once a specified trigger price is reached. Used ma
StopLimit	8	An order that becomes a limit order once the trigger price is reached. It executes only at better.
OCO	32	A conditional order where two linked orders are placed — if one gets executed, the other

SYMPHONY v2.0

4. OrderSide

Enum Name	Numeric Code	Description
`BUY`	49	Indicates a purchase transaction. Used when entering a long position (expecting the pric
`SELL`	50	Indicates a sale transaction. Used when exiting a long position or initiating a short positio

5. TimeInForce

Enum Name	Numeric Code	Description
`DAY`	1	Order remains valid only for the trading day. If not executed by market close, it is cancelled
`GTC`	2	(Good Till Cancelled) Order remains active until it is explicitly cancelled by the trader. May trading sessions.
`IOC`	4	Order must be executed immediately (full or partial). Any unfilled portion is cancelled inst
`GTD`	8	(Good Till Date) Order remains valid until a specified date. If not executed by then, it is can
`EOS`	16	Order remains valid until the end of the trading session (intraday cutoff).
`COL`	32	Order is executed only at the market close price.
`GTDYS`	128	Order remains active for a defined number of trading days.

6. Order Status

Enum Name	Numeric Code	Description
`Open or New`	48	The order has been placed successfully and is waiting for execution.
`PartiallyFilled`	49	Only part of the order quantity has been executed. The remaining is still pending in
`Filled`	50	The entire order quantity has been executed successfully.
`Cancelled`	52	The order has been withdrawn by the trader before it was executed.
`Replaced`	53	The original order has been modified (e.g., price or quantity) and is now treated as a details.
`PendingCancel`	54	A cancel request has been sent but the order has not yet been cancelled.
`Rejected`	56	The order was not accepted due to validation errors, insufficient margin, trading rul

`PendingReplace`	69	A modification request has been sent for the order but confirmation from the excha

7. OrderSource

Enum Name	Numeric Code	Description
`TWS`	1	Orders placed directly from the broker's desktop trading terminal.
`TWSAPI`	2	Orders submitted via the Trader Workstation's API integration, typically through al trading systems.
`WEB`	21	Orders entered manually by the trader through the broker's web-based trading pla
`WEBAPI`	22	Orders placed programmatically via web API integrations — used by custom applic
`MOBILEANDROID`	41	Orders placed through the broker's Android mobile trading application.
`MOBILEIOS`	81	Orders placed through the broker's iOS mobile trading application.
`EnterpriseWeb`	154	Orders originating from an enterprise-grade web trading application.
`EnterpriseMobile`	155	Orders generated from enterprise-level mobile apps built for institutional traders.
`EnterpriseTWS`	156	Orders placed via an advanced or customized version of the Trader Workstation us

8. Position Square Off Mode

Enum Name	Numeric Code	Description
`DayWise`	1	Tracks intraday positions separately for each trading day.
`NetWise`	2	Consolidates positions across multiple days into a net position.

9. Position Square Off Quantity Type

Enum Name	Numeric Code	Description
`Percentage`	0	Exit is defined as a percentage of the total position.
`ExactQty`	1	Exit is defined as a fixed quantity of the position.

10. MarketType

Enum Name	Numeric Code	Description

`Oddlot`	2	Used when the order quantity is less than the standard market lot size. Allows trading quantities of shares.
`Retaildebt`	3	Segment dedicated to retail trading in debt instruments like corporate bonds, governm debentures.
`Auction`	4	Segment where auctions are conducted, usually for settlement shortages or special si auctions when sellers fail to deliver).
`CallAuction1`	5	Call auction mechanism session 1 — typically for illiquid securities. Orders are pooled a uniform price.
`CallAuction2`	6	Call auction mechanism session 2 — another session for specific securities or time slo and executed at a single price.

11. OrderLegStatus

Enum Name	Numeric Code	Description
`SingleOrderLeg`	1	Represents a simple standalone order with no linked legs.
`SpreadFirstLeg`	2	The primary leg of a spread order strategy.
`SpreadSecondLeg`	3	The counter leg of a spread order that offsets the first leg.
`MultilegFirstLeg`	4	The first leg in a multi-leg strategy (often options)
`MultilegSecondLeg`	5	The second leg in a multi-leg strategy.
`MultilegThirdLeg`	6	The third leg in advanced multi-leg strategies.

12. Bo_Leg_Details

Enum Name	Numeric Code	Description
`EntryLeg`	1	The main order leg where the trader enters the position (buy or sell).
`ProfitTargetExitLeg`	2	An automatic exit leg placed along with the entry order to book profit once th
`StoplossExitLeg`	3	A protective exit leg that closes the position if the price moves unfavorably b

13. DayOrNet

Enum Name	Numeric Code	Description

SYMPHONY v2.0

NET	2	Includes delivery trades (CNC) or carry-forward derivative positions (NRML).

14. InstrumentType

Enum Name	Numeric Code	Description
Futures	1	Standardized derivative contracts to buy or sell an asset (stock, index, commodity predetermined price on a future date.
Options	2	Derivative contracts that give the buyer the right, but not the obligation, to buy (C underlying asset at a fixed price within a specified time.
Spread	4	A strategy involving simultaneous buying and selling of related instruments.
Equity	8	Shares of listed companies traded in the cash market segment.
Spot	16	Trades for immediate delivery and settlement at the current market price, typically commodities.
PreferenceShares	32	A class of shares that provides fixed dividends and has priority over equity shares liquidation, but usually with limited voting rights.
Debentures	64	Long-term debt instruments issued by companies or governments to raise funds. payments.
Warrants	128	Financial instruments that give the holder the right to buy equity shares at a speci date.
Miscellaneous	256	Category for instruments that do not fall into standard classifications (used for sp
MutualFund	512	Units of pooled investment schemes managed by Asset Management Companies

15. TradingSession

Enum Name	Numeric Code	Description
PreOpenStart	0	The beginning of the pre-open session where orders can be placed but are not mat
PreOpenEnd	1	The end of the pre-open session when accumulated orders are matched, and the o
NormalStart	2	The start of the regular trading session where continuous matching of buy and sell
NormalEnd	4	The close of the regular trading session; after this point, no further trades are exec
PreClosingStart	8	The start of the pre-closing session, where the system collects data to calculate th auction).

SYMPHONY v2.0

16. ValuationType

Enum Name	Numeric Code	Description
`NONE`	0	Default flag when no special settlement type is applied. The order follows the normal ma
`CL`	1	Orders specifically placed to be executed at the market's closing price during the closin
`LT`	2	Segment used for block deals, bulk trades, or special limited trading windows defined by

17. HoldingType

Enum Name	Numeric Code	Description
`T1_Holdings`	1	Securities bought but not yet fully settled (i.e., on T+1 day).
`Holdings`	3	Fully settled securities that are credited to the client's Demat account.
`MTFHoldings`	3	Securities purchased under Margin Trading Facility where part of the funds are borro

18. StatisticsLevel

Enum Name	Numeric Code	Description
`ParentLevel`	0	Represents the top-level entity in a hierarchy.
`ChildLevel`	1	Represents a sub-entity under a parent.

19. Socket Event

Enum Name	Description
`joined`	Sent when a client successfully connects and joins the WebSocket channel. Confirms that the re
`error`	Indicates a failure or issue (e.g., invalid request, authentication failure, or system error). Requires
`warning`	Non-critical issue or advisory message (e.g., rate-limit approaching, deprecated API usage).
`success`	Confirms that a client request was processed successfully (e.g., order placement acknowledged)
`order`	Event carrying real-time updates about an order lifecycle — placed, modified, filled, cancelled, or r
`trade`	Sent when an order results in an actual trade execution (full or partial). Contains trade ID, price, a

`position`	Real-time updates about portfolio positions, including intraday (DayWise) or carry-forward (NetW
`tradeConversion`	Event triggered when an order/trade is converted from one product type to another (e.g., Intrada
`gttOrder`	Event update for GTT orders (orders that activate only when a trigger price is met).
`gttOrderRejection`	Notification when a GTT order fails to activate due to invalid conditions, margin shortfall, or excha

20. Order Session Type

Enum Name	Numeric Code	Description
`AMO`	1	Standard market order placed during regular market hours.
`OT`	2	*After Market Order* – Orders placed after market hours to be processed when the ma
`INSTI`	4	*Institutional* – Orders placed through institutional trading sessions.
`GTT`	5	Good Till Triggered – Order remains pending until the trigger condition is met.

Data Types & Business Rules

All data types now use business-oriented names, specify allowed sizes, and clearly state any validation rules.

Field Name	Data type	Size	Description
ClientID	`String`	15	Client ID Mandatory in case of Dealer
UserID	`String`	15	UserID of investor client
ExchangeSegment	`String`	15	Exchange Segment
ExchangeInstrumentID	`Integer`	10	Exchange Scrip code or Symbol Token is unique identifier
ProductType	`String`	4	ProductType
OrderType	`String`	10	OrderType
OrderSide	`String`	4	OrderSide
TimeInForce	`String`	5	TimeInForce
Quantity	`Integer`	10	Quantity to transact. In terms of Lots.

OrderUniqueIdentifier	String	20	Echo back to identify order
AppOrderID	Integer	10	Unique Order ID as received in Order Entry Response
AccessPassword	String	20	Access Password of a server
Version	String	20	API Version number of the server
UniqueKey	String	50	The UniqueKey is generated by the server during the HostLookUp
ConnectionString	String	200	IP address of the server to which the client needs to connect.
Remarks	String	50	Any remarks added by the server in the HostLookUp API.
ExchangeOrderID	String	20	It is unique OrderID generated by exchange.
OrderReferenceID	String	20	It is unique OrderReferenceID to identify more than 1 leg orders.
IsSpread	Boolean	5	Indicates Spread order or normal order.
OrderCategoryType	String	10	It represents order market type i.e MarketType
GeneratedBy	String	15	It represents Source from which operation has been performed
MessageCode	String	10	API MessageCode
MessageVersion	String	10	API Message Version
TokenID	String	10	API TokenID
ApplicationType	String	15	API ApplicationType
SequenceNumber	String	15	API SequenceNumber
Timestamps	String	20	
OrderStatus	String	20	OrderStatus
RejectReason	String	500	It is reason if order rejected or canceled.
OrderExpiryDate	String	20	It is the expiry date of order when validity is set to GTD (Good Till D
EntryAppOrderID	Integer	10	The unique order ID for the main/entry order.
ExitAppOrderID	Integer	10	The unique order ID for the linked exit order (Stop-Loss or Target).
OrderLegStatus	String	20	OrderLegStatus

SecretKey	String	20	SecretKey to validate user.
AppKey	String	20	The API App key.
Token	String	500	The authentication token used with every subsequent request.
SquarOff	Integer	10	Represents the profit-taking offset value.
TrailingStoploss	Integer	10	Incremental value (in points or price units) that moves your stop-lo
StopLossPrice	Double	(15,4)	Price in Rupees. Up to 4 decimal places.
ApiOrderSource	String	20	API Order Source name to tag a particular source.
IsInvestorClient	Boolean	5	Represents an investor client; otherwise, represents a dealer.
IsOneTouchUser	Boolean	5	Indicates whether One Touch is enabled for this user.
ExecutionID	String	10	Execution ID is a unique identifier assigned to each trade executio
ExecutionReportIndex	Integer	2	Execution Report Index is a sequential identifier that indicates the for a particular order.
ISIN	String	10	The standard ISIN representing stocks listed on multiple exchange
RMSHoldingId	Integer	10	Unique holding ID.
HoldingType	Integer	15	Type of Holdings
ValuationType	Integer	5	ValuationType
Haircut	Integer	10	Haircut refers to the percentage reduction applied to the market v calculating its collateral value.
CreatedBy	String	20	User or system identifier that created the holding record.
CreatedOn	String	20	Date and time when the holding record was first created.
LastUpdatedBy	String	20	User or system identifier that last modified or updated the holding
IsCollateralHolding	Boolean	5	Indicates whether the holding is being used as collateral for margir
IsBuyAvgPriceProvided	Boolean	5	Indicates whether the buy average price for the holding has been p
IsNeedToDelete	Boolean	5	Indicates whether the holding record needs to be deleted or marke system.
LastUpdatedOn	String	20	The date and time when the holding record was last modified or up
Marketlot	Integer	5	Is the minimum number of shares or units of a security that can be

Amount	Double	(15,4)	Total buy value of position in rupees.
UnrealizedMTM	Double	(15,4)	It's unrealized profit or loss which has not been booked by client.
RealizedMTM	Double	(15,4)	It's realized profit or loss which has been booked by client.
MTM	Double	(15,4)	Mark to market returns (computed based on the last close and the
SumOfQuantityAndPrice	Double	(15,4)	Total buy or sell value of position in rupees.
StatisticsLevel	String	15	StatisticsLevel
IsInterOpPosition	Boolean	5	Interoperability is enabled or disabled.
BEP	Double	(15,4)	It's break even point of position.
IsDayWise	Boolean	5	IsDayWise position conversion.
BroadcastMessage	String	500	Message sent by exchange.
LimitHeader	String	100	Header message used in limits.
CashAvailable	Double	(15,4)	Total cash available for trading.
Collateral	Double	(15,4)	Value of pledged securities/collateral.
MarginUtilized	Double	(15,4)	Margin currently blocked/used for trades.
NetMarginAvailable	Double	(15,4)	Remaining margin after utilization (cashAvailable + collateral - mar
CashMarginAvailable	Double	(15,4)	Actual cash margin balance available.
AdhocMargin	Double	(15,4)	Temporary/adhoc margin assigned by broker/RMS.
NotionalCash	Double	(15,4)	Virtual/notional cash margin (non-cash benefit).
PayInAmount	Double	(15,4)	Funds incoming from client (pending pay-in).
PayOutAmount	Double	(15,4)	Funds payable back to client (pending pay-out).
CNCSellBenefit	Double	(15,4)	Margins benefit from CNC (Cash & Carry) sell transactions.
DirectCollateral	Double	(15,4)	Margin directly from pledged collateral.
HoldingCollateral	Double	(15,4)	Margins benefit from stock holdings.
ClientBranchAdhoc	Double	(15,4)	Branch-level adhoc margin applied.
SellOptionsPremium	Double	(15,4)	Premium received from option selling credited as margin.
TotalBranchAdhoc	Double	(15,4)	Total adhoc margin provided at branch level.

AdhocCurrencyMargin	Double	(15,4)	Temporary margin for currency derivatives.
AdhocCommodityMargin	Double	(15,4)	Temporary margin for commodities.
GrossExposureMarginPresent	Double	(15,4)	Margin blocked for gross exposure positions.
BuyExposureMarginPresent	Double	(15,4)	Margin blocked for buy exposure.
SellExposureMarginPresent	Double	(15,4)	Margin blocked for sell exposure.
VarELMarginPresent	Double	(15,4)	Margin blocked under VaR & ELM requirement.
ScripBasketMarginPresent	Double	(15,4)	Margin blocked for scrip basket restrictions.
GrossExposureLimitPresent	Double	(15,4)	Limit consumed against gross exposure.
BuyExposureLimitPresent	Double	(15,4)	Limit consumed for buy exposure.
SellExposureLimitPresent	Double	(15,4)	Limit consumed for sell exposure.
CNCLimitUsed	Double	(15,4)	Limit used under CNC products.
CNCAmountUsed	Double	(15,4)	Amount utilized under CNC trades.
MarginUsed	Double	(15,4)	Total margin currently blocked/used.
LimitUsed	Double	(15,4)	Other limits consumed (if applicable).
TotalSpanMargin	Double	(15,4)	Margin utilized as SPAN (F&O requirement).
ExposureMarginPresent	Double	(15,4)	Exposure margin used (non-SPAN component).
CNCLimit	Double	(15,4)	Maximum CNC (delivery) trading limit assigned.
TurnoverLimitPresent	Double	(15,4)	Turnover limit allocated.
MTMLossLimitPresent	Double	(15,4)	Limit on MTM loss allowed.
BuyExposureLimit	Double	(15,4)	Maximum exposure allowed for buy trades.
SellExposureLimit	Double	(15,4)	Maximum exposure allowed for sell trades.
GrossExposureLimit	Double	(15,4)	Overall gross exposure limit assigned.
GrossExposureDerivativesLimit	Double	(15,4)	Gross exposure limit for derivatives.
BuyExposureFuturesLimit	Double	(15,4)	Exposure limit for futures buy trades.
BuyExposureOptionsLimit	Double	(15,4)	Exposure limit for options buy trades.
SellExposureOptionsLimit	Double	(15,4)	Exposure limit for options sell trades.

ClientName	String	15	User login ID.
EmailId	String	50	Registered email address of the client.
MobileNo	String	10	Registered mobile number of the client.
PAN	String	15	Client's Permanent Account Number (for compliance/KYC).
IncludeInAutoSquareoff	Boolean	5	Whether the client's positions are eligible for auto square-off.
IncludeInAutoSquareoffBlocked	Boolean	5	Whether auto square-off is blocked for the client.
IsProClient	Boolean	5	Indicates if the client is a proprietary client.
ResidAddress	String	500	Client's residential address.
OfficeAddress	String	500	Client's registered office address.
AccountNumber	String	15	Bank account number.
AccountType	String	10	Type of account ("Savings", "Current", etc.).
BankName	String	100	Name of the bank.
BankBranchName	String	100	Bank branch name.
BankCity	String	50	City where the bank branch is located.
CustomerId	String	15	Customer identifier as provided by the bank.
BankCityPincode	String	10	Pincode of the bank's city.
BankIFSCCode	String	15	IFSC code for the bank branch.
ExchangeSegNumber	Integer	2	Exchange Segment
Enabled	Boolean	5	Whether this exchange segment is active for the client.
ParticipantCode	String	15	Participant/broker code provided by the exchange
IsValid	Boolean	5	Indicates whether the order is valid based on margin checks. true r
MarginRequired	Double	(15,4)	Total margin required (in currency units) to place the order
MarginAvailable	Double	(15,4)	Total margin currently available in the user's account
MarginShortfall	Double	(15,4)	Margin deficit amount if available margin is insufficient
ErrorMessage	Double	500	Error message when IsValid = false. Empty string means no error

SYMPHONY v2.0

The user needs to log in using the appKey and secret key which are generated while registering with the API dashboard along with accessToken & uniqueKey obtained from below steps. The appKey will be first authenticated by the brokers gateway before redirecting to the API service.

OAuth Login Flow

The login flow starts by navigating to the login endpoint.

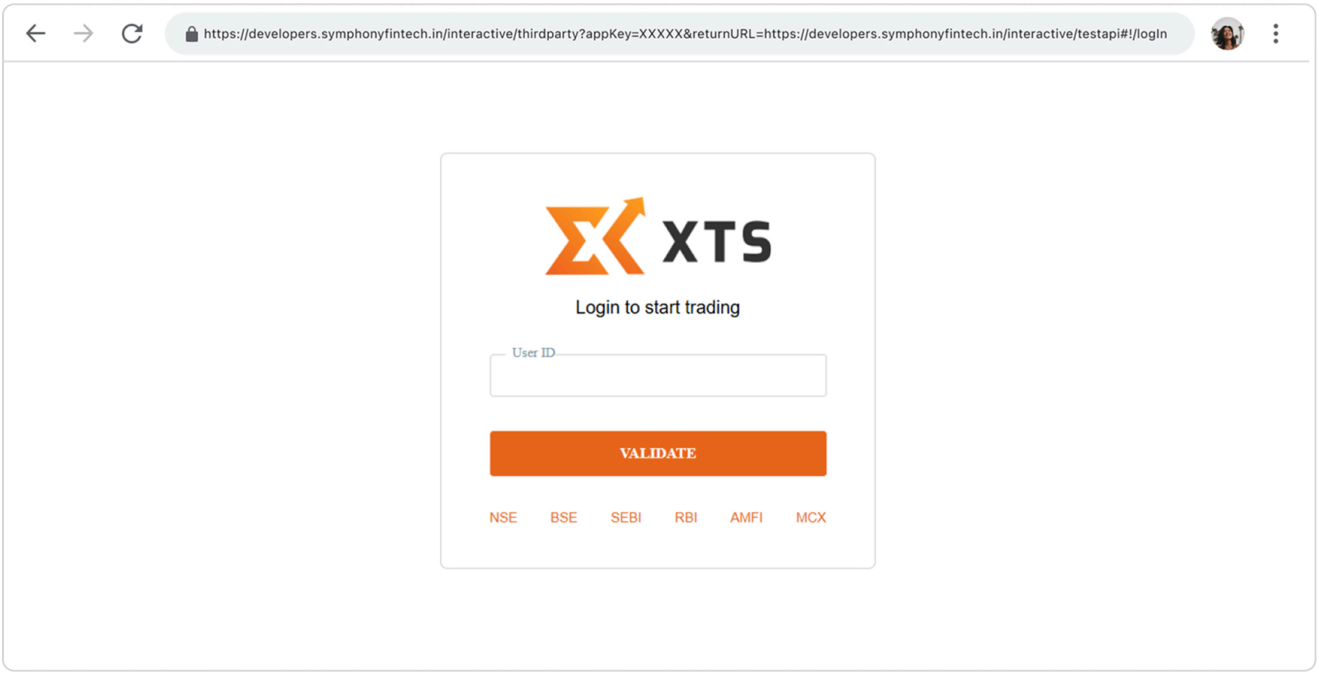
```
https://developers.symphonyfintech.in/1interactive/thirdparty?appKey=XXXX&retu
rnURL=https://developers.symphonyfintech.in/1interactive/testapi#!/logIn
```

Copy

A successful login comes back with an accessToken as a URL query parameter to the return URL.

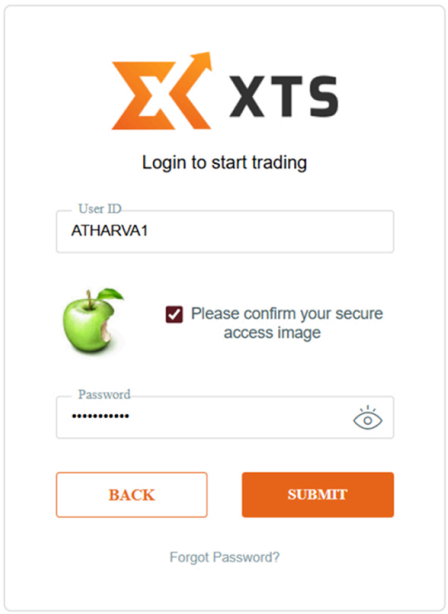
Steps to retrieve accessToken:

1 Navigate to the above URL for login & enter valid UserID.

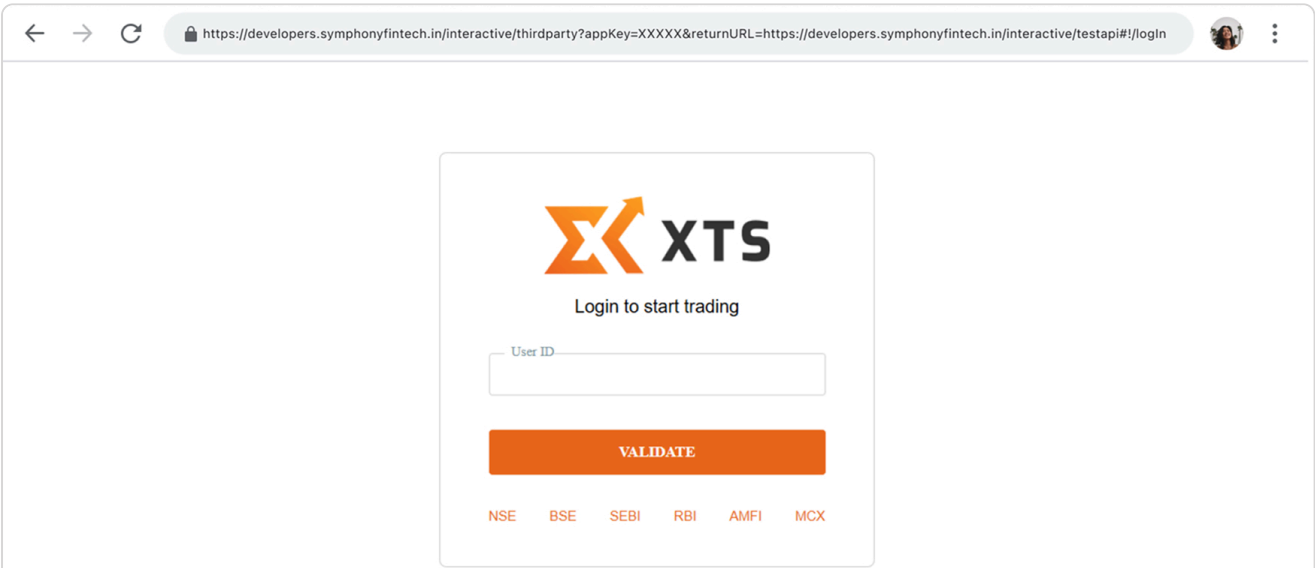


2 Validate the UserID with password.

SYMPHONY v2.0

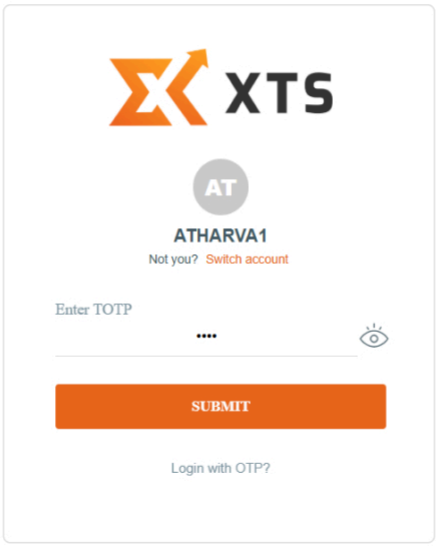


3 Navigate to the above URL for login & enter valid UserID.



4 Once UserID is validated it will ask for pin/otp.

SYMPHONY v2.0



5 **After successfull authorization, accessToken will be recieved on returnURL.**

6 **After retrieving accessToken from Oauth login flow, proceed with Hostlookup to obtain connectionString & UniqueKey**

7 **This accessToken along with appKey, secretKey & uniqueKey will be used in the login API to obtain a session token, which is then used for signing all subsequent requests.**

In summary:

1. Navigate to the provided url for Oauth login flow.
2. A successful login comes back with a accessToken to the return URL.
3. NUsing Hostlookup step, obtain connectionString & UniqueKey.
4. POST the appKey, secretKey, accessToken and uniqueKey to login API.
5. Obtain the session token and use that with all subsequent requests.

HostLookup Login (POST)

SYMPHONY v2.0

- If the provided access password and version are valid, the consumer will receive a UniqueKey and ConnectionString in the response.
- This initial request to the HostLookUp service is required to obtain the UniqueKey and ConnectionString of the Interactive API server. use these values for all subsequent connections.

URL

POST `https://developers.symphonyfintech.in/hostlookup` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
AccessPassword	AccessPassword	Y	The UniqueKey is generated by the server during the Hos
Version	Version	Y	Api Version number of the server.

Request Body JSON

```
{
  "AccessPassword": "2021HostLookUpAccess",
  "version": "interactive_1.0.1"
}
```

Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
uniqueKey	UniqueKey	The UniqueKey is generated by the server during the HostLookUp
connectionString	ConnectionString	IP address of the server to which the client needs to connect.
timeStamp	Timestamps	Timestamp of the server when the UniqueKey is returned to the us
remarks	Remarks	Any remarks added by the server in the HostLookUp API.

Response Body JSON

```
{
  "type": "success",
  "code": "hostlookup",
```

SYMPHONY v2.0


```
"UniqueKey": "U+UTBgtksswxyU0gRFCh2vsq5M2Ft+qVV",
"connectionString": "https://developers.symphonyfintech.in/1interactive",
"timeStamp": "0",
"remarks": ""
```


Copy Show Full

Response Body JSON (Failure)


```
{
  "type": "string",
  "code": "string",
  "description": "Reason for failed reques"
}
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/hostlookup' \
--header 'Content-Type: application/json' \
--data '{
  "accesspassword": "2021HostLookUpAccess",
  "version": "interactive_1.0.2"
}'
```

Copy Show Full

Login API (POST)

The API consumer sends a login request to the Interactive API server with the secretKey and appKey. Upon successful validation, the server returns a login response. Retrieve the token from this response, which must be included in the headers of all subsequent requests.

URL

POST `https://developers.symphonyfintech.in/1interactive/user/session` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
secretKey	SecretKey	Y	The predefined secret key assigned to the clie

SYMPHONY v2.0

Request Body JSON


```
{
  "secretKey": "Doqg107#6V",
  "appKey": "5a75a8676cabe678",
  "uniqueKey": "vp67Z9sQqVdgf2TwYpv9mHY+DH6r3whziBG3djqCcPVVCUyHiwtvhGOq3iCxaGaGvjc0pVsDmT",
  "accessToken": "dd273f43805a88d5acc32ccc38f6844d1bebbb1b384292e4739e4dc5059a515"
}
```


Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
Enmus	Object	Object of all Enums i.e socketEvent , OrderSide , positionSqureOffMode,positionSquareOffQuantityType,dayOrNet, exchangeSegment,instrumentType, orderSource,exchangeInfo,product
clientCodes	Array	If isInvestorClient is false, an array list of clients mapped to the dealer is r ClientID is returned as the investor client.
exchangeSegmentArray	Array	ExchangeSegment
token	Token	The authentication token used with every subsequent request, unless it API.
userID	UserID	userID of investor client
isInvestorClient	isInvestorClient	If the value is true, it represents an investor client; otherwise, it represen
isOneTouchUser	IsOneTouchUser	Indicates whether One Touch is enabled for this user.

Response Body JSON


```
{
  "type": "success",
  "code": "s-user-0001",
  "description": "Valid User.",
  "result": {
  "enums": {
  "socketEvent": [
        "joined",
        "error",
        "warning"
```

SYMPHONY v2.0

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/user/session' \
--header 'Content-Type: application/json' \
--data '{
  "secretKey": "Doqg107#6V",
  "appKey": "5a75a8676cabe678",
  "uniqueKey": "vp67Z9sQqVdgf2TwYpv9mHY+DH6r3whziBG3djqCcPVVCUyHiwtvhGOq3iCxaGaGvjc0pVsDmT",
  "accessToken": "dd273f43805a88d5acc32ccc38f6844d1bebbb1b384292e4739e4dc5059a515"
}'
```

Copy | Show Full

Session Logout (Delete)

This call invalidates the session token and terminates the API session. After this, the user must go through the login flow again and retrieve a new session token from the login response before performing further activities. This does not log the user out of the XTS TWS application.

URL

DELETE https://developers.symphonyfintech.in/1interactive/user/session Copy

Response Body Parameters

```
{
  "type": "success",
  "code": "s-user-0001",
  "description": "successfully logout from all API.",
  "result": ""
}
```

Copy | Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location --request DELETE 'https://developers.symphonyfintech.in/1interactive/user/session'
--header 'Authorization: xxxxxx'
```

SYMPHONY v2.0

Get Balance (GET)

Retrieves balance-related information for a user. The response includes details about equities, derivatives, upfront margins, available exposure, and other RMS-related balances. This API is primarily used to check real-time trading limits and exposures.

URL

GET `https://developers.symphonyfintech.in/interactive/user/balance?clientID=SYMP` Copy

Request Body Parameters

Parameter Name	Type	Description	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer

Response Body Parameters

Parameter Name	Type	Description
BalanceList	BalanceList	-
limitHeader	LimitHeader	A pipe (|) separated string, often representing cate segmentlexchangelproduct (here "ALLIALLIALL" =
limitObject	limitObject	Main object holding multiple limit & margin details.
RMSSubLimits	RMSSubLimitsList	Represents RMS (Risk Management System) **sub-li**
cashAvailable	CashAvailable	Total free cash available for trading.
collateral	Collateral	Value of pledged securities/collateral.
marginUtilized	MarginUtilized	Margin currently blocked/used for trades.
netMarginAvailable	NetMarginAvailable	Remaining margin after utilization (cashAvailable +
MTM	MTM	Mark-to-Market profit/loss.
UnrealizedMTM	UnrealizedMTM	P/L from open positions not yet squared off.
RealizedMTM	RealizedMTM	P/L from closed positions.
MarginAvailable	MarginAvailableList	Represents types of available margins.
CashMarginAvailable	CashMarginAvailable	Actual cash margin balance available.

NotinalCash	NotinalCash	Virtual/notional cash margin (non-cash benefit).
PayInAmount	PayInAmount	Funds incoming from client.
PayOutAmount	PayOutAmount	Funds payable back to client.
CNCSellBenifit	CNCSellBenifit	Margins benefit from CNC (Cash & Carry) sell trans
DirectCollateral	DirectCollateral	Margin directly from pledged collateral.
HoldingCollateral	HoldingCollateral	Margins benefit from stock holdings.
ClientBranchAdhoc	ClientBranchAdhoc	Branch-level adhoc margin allocated.
SellOptionsPremium	SellOptionsPremium	Premium received from option selling credited as m
TotalBranchAdhoc	TotalBranchAdhoc	Total adhoc margin provided at branch level.
AdhocFOMargin	AdhocFOMargin	Temporary margin for Futures & Options.
AdhocCurrencyMargin	AdhocCurrencyMargin	Temporary margin for currency derivatives.
AdhocCommodityMargin	AdhocCommodityMargin	Temporary margin for commodities.
marginUtilizedList	marginUtilizedList	Represents how margin is being consumed.
GrossExposureMarginPresent	GrossExposureMarginPresent	Margin blocked for gross exposure positions.
BuyExposureMarginPresent	BuyExposureMarginPresent	Margin blocked for buy exposure.
SellExposureMarginPresent	SellExposureMarginPresent	Margin blocked for sell exposure.
VarELMarginPresent	VarELMarginPresent	Margin blocked under VaR + ELM requirement.
ScripBasketMarginPresent	ScripBasketMarginPresent	Margin blocked for scrip basket restrictions.
GrossExposureLimitPresent	GrossExposureLimitPresent	Limit consumed against gross exposure.
BuyExposureLimitPresent	BuyExposureLimitPresent	Limit consumed for buying exposure.
SellExposureLimitPresent	SellExposureLimitPresent	Limit consumed for selling exposure.
CNCLimitUsed	CNCLimitUsed	Limit used under CNC products.
CNCAamountUsed	CNCAamountUsed	Amount utilized under CNC trades.
MarginUsed	MarginUsed	Total margin currently blocked/used.
LimitUsed	LimitUsed	Other limits consumed (if applicable).

SYMPHONY v2.0

ExposureMarginPresent	ExposureMarginPresent	Exposure margin used (non-SPAN component).
limitsAssigned	limitsAssignedList	Represents maximum limits allocated to accounts.
CNCLimit	CNCLimit	Maximum CNC (delivery) trading limit assigned.
TurnoverLimitPresent	TurnoverLimitPresent	Turnover limit allocated.
MTMLossLimitPresent	MTMLossLimitPresent	Limit on MTM loss allowed.
BuyExposureLimit	BuyExposureLimit	Maximum exposure allowed for buy trades.
SellExposureLimit	SellExposureLimit	Maximum exposure allowed for sell trades.
GrossExposureLimit	GrossExposureLimit	Overall gross exposure limit assigned.
GrossExposureDerivativesLimit	GrossExposureDerivativesLimit	Gross exposure limit for derivatives.
BuyExposureFuturesLimit	BuyExposureFuturesLimit	Exposure limit for futures buy trades.
BuyExposureOptionsLimit	BuyExposureOptionsLimit	Exposure limit for options buy trades.
SellExposureOptionsLimit	SellExposureOptionsLimit	Exposure limit for options sell trades.
SellExposureFuturesLimit	SellExposureFuturesLimit	Exposure limit for futures sell trades.
AccountID	AccountID	User login ID.

Response Body JSON


```
{
  "type": "success",
  "code": "s-user-0002",
  "description": "OK",
  "result": {
    "BalanceList": {
      "limitHeader": "ALL|ALL|ALL",
      "limitObject": {
        "RMSSubLimits": {
          "cashAvailable": 99999999999999
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

SYMPHONY v2.0

Copy Show Full

Get Profile (GET)

Using the session token, the user can access their profile stored with the broker. The profile can be retrieved at any point in time.

URL

GET `https://developers.symphonyfintech.in/1interactive/user/profile?clientID=SYMP` Copy

Request Body Parameters

Parameter Name	Type	Description	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer

Response Body Parameters

Parameter Name	Type	Description
ClientId	ClientID	User login ID.
ClientName	ClientName	Registered name of the client.
EmailId	EmailId	Registered email address of the client.
MobileNo	MobileNo	Registered mobile number of the client.
PAN	PAN	Client's Permanent Account Number (for complian
IsClientAutoSquareoff	IncludeInAutoSquareoffBlocked	Whether auto square-off is blocked for the client.
IncludeInAutoSquareoffBlocked	IncludeInAutoSquareoffBlocked	Whether auto square-off is blocked for the client.
IsProClient	IsProClient	Indicates if the client is a proprietary client.
IsInvestorClient	IsInvestorClient	Indicates if the client is categorized as an investor
ResidentialAddress	ResidentialAddress	Client's registered residential address.
OfficeAddress	OfficeAddress	Client's registered office address.
ClientBankInfoList	ClientBankInfoList	Bank account details linked to the client.

SYMPHONY v2.0

AccountNumber	AccountNumber	Bank account number.
AccountType	AccountType	Type of account ("Savings", "Current", etc.).
BankName	BankName	Name of the bank.
BankBranchName	BankBranchName	Bank branch name.
BankCity	BankCity	City where the bank branch is located.
CustomerId	CustomerId	Customer identifier as provided by the bank.
BankCityPincode	BankCityPincode	Pincode of the bank's city.
BankIFSCCode	BankIFSCCode	IFSC code for the bank branch.
ClientExchangeDetailsList	ClientExchangeDetailsList	Exchange-specific details mapped to the client. Ea BSEFO, etc.) will appear as a nested object.
ClientId	ClientID	User login ID.
ExchangeSegNumber	ExchangeSegNumber	Segment number used internally to identify the ex
Enabled	Enabled	Whether this exchange segment is active for the c
ParticipantCode	ParticipantCode	Participant/broker code provided by the exchange

Response Body JSON


```
{
  "type": "success",
  "code": "s-user-0001",
  "description": "User profile",
  "result": {
    "ClientId": "SYMP1",
    "ClientName": "SYMP",
    "EmailId": "symp.t@symphonyfintech.com",
    "MobileNo": "5555587878",
    "PAN": "HSDIG2342J"
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/user/profile?clientID=SYMP' \
--header 'Authorization: xxxxxx'
```

SYMPHONY v2.0

Brokerage (POST)

Using the session token, the user can access their profile stored with the broker. The profile can be retrieved at any point in time.

URL

POST https://developers.symphonyfintech.in/1interactive/user/calculatebrokerage Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**Y**	CLIENT ID
exchangeSegment	ExchangeSegment	**Y**	Exchange Scrip code or Symbol Token i
exchangeInstrumentID	ExchangeInstrumentID	**Y**	Exchange Scrip code or Symbol Token i
productType	ProductType	**Y**	ProductType
orderSide	OrderSide	**Y**	OrderSide
orderQty	OrderQuantity	**Y**	Quantity to transact. In terms of Lots
orderPrice	Price	**Y**	The price to execute the order at

Request Body JSON

```
{
  "clientID": "SYMP",
  "brokerageOrderInformation": [
    {
      "exchangeSegment": 2,
      "exchangeInstrumentID": 37054,
      "productType": "NRML",
      "orderSide": "BUY",
      "orderQty": 75,
      "orderPrice": 25000
```

Copy Show Full

Response Body Parameters

Parameter Name	Type	Description

SYMPHONY v2.0

OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is sent in rec
Brokerage	Brokerage	Brokerage value
STTOrCTT	STTOrCTT	Securities Transaction Tax
ExchangeTurnoverCharges	ExchangeTurnoverCharges	Exchange turnover fees
ExchangeCharges	ExchangeCharges	Exchange transaction char
GST	GST	Goods and Services Tax
SebiCharges	SebiCharges	SEBI regulatory charges
StampDuty	StampDuty	Stamp duty on trade
ClearingCharges	ClearingCharges	Clearing corporation fees
DPCharges	DPCharges	Depository participant cha
Total	Total	Total charges

Response Body JSON


```
{
  "type": "success",
  "code": "s-calculatebrokerage-0003",
  "description": "Calculate Brokerage Data",
  "result": {
    "brokerageDeatils": {
      "Brokerage": 37.5,
      "STTOrCTT": 187500,
      "ExchangeTurnoverCharges": 187.5,
      "ExchangeCharges": 18.75
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/user/calculatebrokerage' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "clientID": "SYMP",
```

SYMPHONY v2.0

```
"exchangeSegment": 2,
"exchangeInstrumentID": 47664,
```

Copy Show Full

Standard Orders

Place Order API (POST)

Order Placement Workflow & Key Considerations

- Read all ENUM values from the login response along with the **authentication token**
- Use the ENUM values and token when **constructing the Place Order request**
- The **token must be included in the JSON request headers for all API calls** for all API calls
- The **Place Order request is asynchronous** in nature
- On submission, the user receives an **appOrderID**
- The **appOrderID** can be used later with the **Order Book (Get Order) API** to confirm order execution.
- The **execution of an order** depends on multiple factors: (1.RMS limits, 2.Risk checks, 3.Other validations)
- **Limit orders** may remain **open throughout the trading day** if not executed.

URL

POST `https://developers.symphonyfintech.in/1interactive/orders` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
exchangeSegment	ExchangeSegment	Y	ExchangeSegment
exchangeInstrumentID	ExchangeInstrumentID	Y	Exchange Scrip code or Symbol Token is unique ident
productType	ProductType	Y	ProductType
orderType	OrderType	Y	OrderType
orderSide	OrderSide	Y	OrderSide
timeInForce	TimeInForce	Y	TimeInForce
disclosedQuantity	OrderQuantity	Y	Quantity to disclose (for equity)
orderQuantity	OrderQuantity	Y	Quantity to transact. In terms of Lots

SYMPHONY v2.0

stopPrice	Price	Y	The price at which an order should be triggered (SL, S
orderUniqueIdentifier	OrderUniqueIdentifier	N	Echo back to identify order
clientID	ClientID	N	ClientID Mandatory in case of Dealer
apiOrderSource	ApiOrderSource	N	API Order Source can be a third party application nam your order, which will be used to track your order with

Request Body JSON


```
{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": 3045,
  "productType": "NRML",
  "orderType": "LIMIT",
  "orderSide": "BUY",
  "timeInForce": "DAY",
  "disclosedQuantity": 0,
  "orderQuantity": 15,
  "limitPrice": 254.55
```


Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
AppOrderID	AppOrderID	Unique order ID
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is send in request bo

Response Body JSON


```
{
  "type": "success",
  "code": "s-orders-0001",
  "description": "Request sent",
  "result": {
    "AppOrderID": 2190766863,
    "OrderUniqueIdentifier": "123abc",
    "ClientID": "SYMP1"
  }
}
```

SYMPHONY v2.0

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders' \
--header 'Authorization: xxxxxxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": 2885,
  "productType": "NRML",
  "orderType": "LIMIT",
  "orderSide": "BUY",
  "timeInForce": "DAY"
```

Copy | Show Full

Modify Order API (PUT)

XTS provides the ability to modify open orders by allowing you to change a limit order to a market order (or vice versa), update the price or quantity of an open limit order, and modify the disclosed quantity or stop-loss of any open stop-loss order.

Note : XTS considers open orders whose order status is either New, Replaced, PartiallyFilled.

URL

PUT `https://developers.symphonyfintech.in/1interactive/orders` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
appOrderID	AppOrderID	Y	It is system generated unique order number
modifiedProductType	ProductType	Y	ProductType
modifiedOrderType	OrderType	Y	OrderType
modifiedDisclosedQuantity	Quantity	Y	Quantity to disclose (for equity)
modifiedOrderQuantity	Quantity	Y	Quantity to transact. In terms of Lots
modifiedLimitPrice	Price	Y	The price to execute the order at
modifiedStopPrice	Price	Y	The price at which an order should be triggered (SL

SYMPHONY v2.0

orderUniqueIdentifier	OrderUniqueIdentifier	**N**	Echo back to identify order
apiOrderSource	ApiOrderSource	**N**	API Order Source can be third party application nan your order, which will be used to track your order wi
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer

Request Body JSON


```
{
  "appOrderID": 2190766863,
  "modifiedProductType": "NRML",
  "modifiedOrderType": "LIMIT",
  "modifiedOrderQuantity": 25,
  "modifiedDisclosedQuantity": 0,
  "modifiedLimitPrice": 255.65,
  "modifiedStopPrice": 0,
  "modifiedTimeInForce": "DAY",
  "orderUniqueIdentifier": "123abc"
```


Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
AppOrderID	AppOrderID	Unique order ID
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is send in request b

Response Body JSON


```
{
  "type": "success",
  "code": "s-orders-0001",
  "description": "Request sent",
  "result": {
    "AppOrderID": 2190766863,
    "OrderUniqueIdentifier": "123abc",
    "ClientID": "SYMP1"
  }
}
```


Copy Show Full

SYMPHONY v2.0

```
curl --location --request PUT 'https://developers.symphonyfintech.in/1interactive/orders' \
--header 'Authorization: xxxxxxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "appOrderID": "10001253450000041478",
  "modifiedProductType": "NRML",
  "modifiedOrderType": "LIMIT",
  "modifiedOrderQuantity": 75,
  "modifiedDisclosedQuantity": 0,
  "modifiedLimitPrice": 28000
```

Copy Show Full

Cancel Order API (DELETE)

This API can be used to cancel any open order by providing the correct appOrderID that matches the selected open order.
Note : XTS considers open orders whose order status is either New, Replaced, PartiallyFilled.

URL

DELETE `https://developers.symphonyfintech.in/1interactive/orders?appOrderID=2190766863` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
AppOrderID	AppOrderID	**Y**	Unique order ID
OrderUniqueIdentifier	OrderUniqueIdentifier	**Y**	Echo back to identify orde
ClientID	ClientID	**N**	ClientID which is send in r

Response Body JSON

```
{
  "type": "success",
  "code": "s-user-0001",
  "description": "Request sent",
  "result": [
    {
      "AppOrderID": 1200043151,
      "ClientID": "SYMP"
    }
  ]
```

SYMPHONY v2.0

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location --request DELETE 'https://developers.symphonyfintech.in/1interactive/orders?appOrde
--header 'Authorization: xxxxxxxxxx' \
--data ''
```

Copy | Show Full

CancelAll Order (POST)

This API can be called for a particular segment to either:

- **Cancel all open orders of the user by passing 0 in ExchangeInstrumentID** in the request.
- **Cancel all open orders for a specific instrument by passing its ExchangeInstrumentID (e.g., 3045)** in the request.

For example:

- If you pass ExchangeSegment: NSECM and ExchangeInstrumentID: 0, it will cancel **all open orders for NSECM** for that user.
- If you pass ExchangeSegment: NSECM and ExchangeInstrumentID: 3045, it will cancel **all open orders for NSECM SBIN (3045)** for that user.

Note : XTS considers open orders whose order status is either New, Replaced, PartiallyFilled.

URL

POST `https://developers.symphonyfintech.in/1interactive/orders/cancelall` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
exchangeSegment	ExchangeSegment	**Y**	ExchangeSegment
exchangeInstrumentID	ExchangeInstrumentID	**Y**	Exchange Scrip code or Symbol Token i
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer

Request Body JSON

SYMPHONY v2.0

```
    "exchangeSegment": "NSECM",
    "exchangeInstrumentID": 3045
  }
]
```

Copy Show Full

Response Body JSON

```
{
  "type": "success",
  "code": "s-cancelAll-0001",
  "description": "Cancel All Order Request Send Successfully",
  "result": ""
}
```

Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/cancelall' \
--header 'Authorization: xxxxxxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": "1333"
}'
```

Copy Show Full

Cover orders (CO)

Place Cover Order (POST)

A Cover Order (CO) is an advanced intraday order that is always accompanied by a **mandatory Stop-Loss Order.** This helps users minimize potential losses by safeguarding against unexpected market movements.

A Cover Order offers **high leverage** and is available in the following segments:

- **Equity Cash**
- **Equity F&O**

SYMPHONY v2.0

A Cover Order consists of **embedded orders:**

- **Limit/Market Order**
  The investor aims to enter a trade at either the best available market price or a desired limit price of the asset at that time.

- **Stop-Loss Order**
  The main order is always accompanied by a **Stop-Loss Order** and the position is **squared off automatically** if the market price reaches the specified stop-loss price.

- The stop-loss order **can be modified** but **cannot be cancelled.**

URL

POST `https://developers.symphonyfintech.in/1interactive/orders/cover` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
exchangeSegment	ExchangeSegment	**Y**	ExchangeSegment
exchangeInstrumentID	ExchangeInstrumentID	**Y**	Exchange Scrip code or Symbol Token is unique ident
orderType	OrderType	**Y**	OrderType
orderSide	OrderSide	**Y**	OrderSide
disclosedQuantity	DisclosedQuantity	**Y**	Quantity to disclose (for equity)
orderQuantity	Quantity	**Y**	Quantity to transact. In terms of Lots
limitPrice	Price	**Y**	The price to execute the order at
stopPrice	Price	**Y**	The price at which an order should be triggered (SL, S
orderUniqueIdentifier	OrderUniqueIdentifier	**N**	Echo back to identify order
apiOrderSource	ApiOrderSource	**N**	API Order Source can be third party application name your order, which will be used to track your order with
clientID	ClientID	**N**	ClientID Mandatory in case of Dealer

Request Body JSON

```
{
  "exchangeSegment": "NSECM",
```

SYMPHONY v2.0


```
"orderQuantity": 15,
"disclosedQuantity": 0,
"limitPrice": 254.55,
"stopPrice": 251.5,
"orderType": "LIMIT",
```


Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
EntryAppOrderID	EntryAppOrderID	The unique order ID for the **main/entry order**
ExitAppOrderID	ExitAppOrderID	The unique order ID for the **linked exit order** (Stop-Los
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is send in request body

Response Body JSON


```
{
  "type": "success",
  "code": "s-orders-0001",
  "description": "Request sent",
  "result": {
    "EntryAppOrderID": 1323797170,
    "ExitAppOrderID": 3727296468,
    "OrderUniqueIdentifier": "123abc",
    "ClientID": "SYMP1"
  }
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java


```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/cover' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": 3045,
  "orderSide": "BUY",
  "orderQuantity": 15,
  "disclosedQuantity": 0,
```

SYMPHONY v2.0

Exit Cover Order (PUT)

The Exit Cover API allows the user to easily exit an open stop-loss order by converting it into an exit order.

URL

PUT https://developers.symphonyfintech.in/1interactive/orders/cover Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
appOrderID	AppOrderID	**Y**	Unique order ID
orderUniqueIdentifier	OrderUniqueIdentifier	**N**	Echo back to identify orc
clientID	ClientID	**N**	Client ID Mandatory in ca

Request Body JSON

```
{
  "appOrderID": 3727296468,
  "OrderUniqueIdentifier": "123abc",
  "ClientID": "SYMP1"
}
```

Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
AppOrderID	AppOrderID	Unique order ID
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is send in request b

Response Body JSON

```
{
  "type": "success",
```

SYMPHONY v2.0


```
  "result": {
    "AppOrderID": 3727296468,
    "ClientID": "SYMP1"
  }
}
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location --request PUT 'https://developers.symphonyfintech.in/1interactive/orders/cover' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "appOrderID": 3727296468,
  "OrderUniqueIdentifier": "123abc",
  "ClientID": "SYMP1"
}'
```

Copy Collapse

Bracket Order

Bracket Order (POST)

Bracket Order (BO) is a type of order where you can enter a new position along with a **target/exit** and a **stop-loss order.**

- As soon as the **main order** is executed, the system will automatically place **two more orders:**
  - A **profit-taking order**
  - A **stop-loss order**
- When either the **profit-taking order** or the **stop-loss order** is executed, the other order is **automatically cancelled.**

Bracket Orders are essentially **algorithmic orders.**

- Entry with Bracket Orders can be done using **Limit Orders** or **Stop-Loss Orders based on triggers.**
- The stop-loss for exit will always be an **SL order.**

You can also use a **Trailing Stop-Loss.** This means:

- If the contract/stock moves in your favor (i.e., the position becomes profitable) by a certain number of ticks, the

SYMPHONY v2.0

Using a trailing stop-loss is **optional**, but if you enable it, you **cannot modify the Bracket Order** after placement.

URL

POST https://developers.symphonyfintech.in/1interactive/orders/bracket Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
exchangeSegment	ExchangeSegment	Y	ExchangeSegment
exchangeInstrumentID	ExchangeInstrumentID	Y	Exchange Scrip code or Symbol Token is unique ident
orderType	OrderType	Y	OrderType
orderSide	OrderSide	Y	OrderSide
disclosedQuantity	Quantity	Y	Quantity to disclose (for equity)
orderQuantity	Quantity	Y	Quantity to transact. In terms of Lots
limitPrice	Price	Y	The price to execute the order at
stopLossPrice	StopLossPrice	Y	A stoploss price is the price in a stop order that trigg order.
squarOff	SquarOff	Y	Represents the profit-taking offset value
trailingStoploss	TrailingStoploss	Y	It is an incremental value (in ticks or price units) that profitable direction
orderUniqueIdentifier	OrderUniqueIdentifier	N	Echo back to identify order
apiOrderSource	ApiOrderSource	N	API Order Source can be third party application name your order, which will be used to track your order with
clientID	ClientID	N	ClientID Mandatory in case of Dealer

Request Body JSON

```
{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": 3045,
  "orderType": "LIMIT",
  "orderSide": "BUY",
  "disclosedQuantity": 0,
  "orderQuantity": 15,
```

SYMPHONY v2.0

Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
AppOrderID	AppOrderID	Unique order ID
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is sent in request b

Response Body JSON

```
{
  "type": "success",
  "code": "s-bracketorders-0002",
  "description": "Request sent",
  "result": {
    "AppOrderID": 1290766863,
    "OrderUniqueIdentifier": "123abc",
    "ClientID": "SYMP1"
  }
}
```

Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/bracket' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": 2885,
  "orderType": "LIMIT",
  "orderSide": "BUY",
  "orderQuantity": 1,
  "disclosedQuantity": 0
```

Copy Show Full

Modify Bracket Order (PUT)

SYMPHONY v2.0

- You can change the **limit price** of an open target order.
- You can change the **stop-loss price** of any open stop-loss order.

However:

- When modifying a **stop-loss order,** the **limit price** will **not** be updated.
- Similarly, when modifying a **profit (target) order,** the **stop-loss price** will **not** be updated.

Note : XTS considers open orders whose order status is either New, Replaced, PartiallyFilled.

URL

PUT `https://developers.symphonyfintech.in/1interactive/orders/bracket` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
appOrderID	AppOrderID	**Y**	Unique order ID
orderQuantity	OrderQuantity	**Y**	Quantity to transact. In terms of Lots
limitPrice	Price	**Y**	The price to execute the order at
stopLossPrice	Price	**Y**	A stop loss price is the price in a stop order that triggers order.
orderUniqueIdentifier	OrderUniqueIdentifier	**N**	Echo back to identify order
apiOrderSource	ApiOrderSource	**N**	API Order Source can be a third party application name order, which will be used to track your order with a parti
clientID	ClientID	**N**	ClientID Mandatory in case of Dealer

Request Body JSON

```
{
  "appOrderID": 1290766863,
  "orderQuantity": 15,
  "limitPrice": 254.55,
  "stopLossPrice": 245.55,
  "orderUniqueIdentifier": "123abc",
  "apiOrderSource": "ThirdPartyAppName",
  "clientID": "SYMP1"
}
```

SYMPHONY v2.0

Response Body Parameters

Parameter Name	Type	Description
AppOrderID	AppOrderID	Unique order ID
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is send in request b

Response Body JSON


```
{
  "type": "success",
  "code": "s-bracketorder-0008",
  "description": "Request sent",
  "result": {
    "AppOrderID": 1290766863,
    "orderUniqueIdentifier": "123abc",
    "ClientID": "SYMP1"
  }
}
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

```
curl --location --request PUT 'https://developers.symphonyfintech.in/1interactive/orders/bracket'
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "limitPrice": "1251",
  "stopLossPrice": "21",
  "orderQuantity": "1",
  "appOrderID": "1800125097"
}'
```

Copy Collapse

Cancel Bracket Order (Delete)

This API can be used to cancel any open Bracket Order by providing the correct BOEntryOrderID that matches the selected open order to be cancelled.

SYMPHONY v2.0

DELETE ntryOrderId =2190766863 Copy

Note : XTS considers open orders whose order status is either New, Replaced, PartiallyFilled.

Request Body Parameters

Parameter Name	Type	Mandatory	Description
boEntryOrderId	AppOrderID	**Y**	Unique order ID
orderUniqueIdentifier	OrderUniqueIdentifier	**N**	Echo back to identify orc
clientID	ClientID	**N**	Client ID mandatory in ca

Response Body Parameters

Parameter Name	Type	Description
AppOrderID	AppOrderID	Unique order ID
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order
ClientID	ClientID	ClientID which is send in request b

Response Body JSON


```
{
  "type": "success",
  "code": "s-bracketorder-0014",
  "description": "Request sent",
  "result": {
    "AppOrderID": 1290766863,
    "ClientID": "SYMP1"
  }
}
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

```
curl --location --request DELETE 'https://developers.symphonyfintech.in/1interactive/orders/bracket
```

SYMPHONY v2.0

Copy Collapse

OrderBook And TradeBook

OrderBook (GET)

The Order Book contains the status of all orders placed by a user. The possible states of an order are referred to as the Order Status.

URL

GET `https://developers.symphonyfintech.in/1interactive/orders?clientID=SYMP1` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer

Response Body Parameters

Parameter Name	Type	Description
LoginID	UserID	User login ID
ClientID	ClientID	User specific identification
AppOrderID	AppOrderID	Unique order ID
OrderReferenceID	OrderReferenceID	It is unique OrderReferenceID to identify more than 1 leg
GeneratedBy	GeneratedBy	It represents Source from which operation has been performed.
ExchangeOrderID	ExchangeOrderID	It is unique OrderID generated by exchange
OrderCategoryType	OrderCategoryType	It represents order market type, i.e. MarketType
ExchangeSegment	ExchangeSegment	ExchangeSegment

OrderSide	OrderSide	OrderSide
OrderType	OrderType	OrderType
ProductType	ProductType	ProductType
TimeInForce	TimeInForce	TimeInForce
OrderPrice	Price	The price to execute the order at
OrderQuantity	Quantity	Quantity to transact
OrderStopPrice	Price	The price at which an order should be triggered (SL, SL-M
OrderStatus	OrderStatus	It is the status of order i.e. OrderStatus
OrderAverageTradedPrice	OrderAverageTradedPrice	Average traded price, also referred as a Volume-Weighte
LeavesQuantity	Quantity	The remaining shares still left to buy/sell of a single instr
CumulativeQuantity	Quantity	The total number of shares bought/sold on a single orde
OrderDisclosedQuantity	Quantity	Quantity to disclose (for equity)
OrderGeneratedDateTime	Timestamps	Timestamp at which the order was registered by the API
ExchangeTransactTime	Timestamps	Timestamp at which the order was registered by the exc
LastUpdateDateTime	Timestamps	Timestamp at which an order's state changed
OrderExpiryDate	OrderExpiryDate	It is order expiry date of order when validity set to GTD (G
CancelRejectReason	RejectReason	It is reason if order rejected or canceled
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order (20 char length)
OrderLegStatus	OrderLegStatus	It represents type of order i.e. OrderLegStatus
BoLegDetails	BoLegDetails	Bracket Order Leg details i.e. BoLegDetails
IsSpread	IsSpread	Indicates Spread order or normal order
BoEntryOrderID	AppOrderID	Unique order ID
MessageCode	MessageCode	API MessageCode

SYMPHONY v2.0

TokenID	TokenID	API TokenID
ApplicationType	ApplicationType	API ApplicationType
SequenceNumber	SequenceNumber	API SequenceNumber

Response Body JSON


```
{
  "type": "success",
  "code": "s-user-0001",
  "description": "Success order book",
  "result": [
    {
      "LoginID": "SYMP1",
      "ClientID": "SYMP1",
      "AppOrderID": 648468730,
      "OrderReferenceID": ""
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders' \
--header 'Authorization:
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJQUkFNT0QxIiwibWVtYmVySUQiOiJEMSIsInNvdXJjZSI6I
```

Copy Show Full

TradeBook (GET)

The trade book returns a list of all trades executed on a particular day that were placed by the user. It displays all filled and partially filled orders.

URL

GET http://160.30.125.86:10955/interactive/orders/trades Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description

SYMPHONY v2.0

Response Body Parameters

Parameter Name	Type	Description
LoginID	UserID	User login ID
ClientID	ClientID	User specific identification
AppOrderID	AppOrderID	Unique order ID
OrderReferenceID	OrderReferenceID	It is unique OrderReferenceID to identify more than 1 leg or
GeneratedBy	GeneratedBy	It represents Source from which operation has been performed.
ExchangeOrderID	ExchangeOrderID	It is unique OrderID generated by exchange
OrderCategoryType	OrderCategoryType	It represents order market type, i.e. MarketType
ExchangeSegment	ExchangeSegment	ExchangeSegment
ExchangeInstrumentID	ExchangeInstrumentID	Exchange Scrip code or Symbol Token is unique identifier
OrderSide	OrderSide	OrderSide
OrderType	OrderType	OrderType
ProductType	ProductType	ProductType
TimeInForce	TimeInForce	TimeInForce
OrderPrice	Price	The price to execute the order at
OrderQuantity	Quantity	Quantity to transact
OrderStopPrice	Price	The price at which an order should be triggered (SL, SL-M)
OrderStatus	OrderStatus	It is the status of order i.e. OrderStatus
OrderAverageTradedPrice	Price	Average traded price, also referred as Volume-Weighted A
LeavesQuantity	Quantity	The remaining shares still left to buy/sell of a single instru
CumulativeQuantity	Quantity	The total number of shares bought/sold on a single order

OrderGeneratedDateTime	Timestamps	Timestamp at which the order was registered by the API
ExchangeTransactTime	Timestamps	Timestamp at which the order was registered by the excha
LastUpdateDateTime	Timestamps	Timestamp at which an order's state changed
OrderExpiryDate	OrderExpiryDate	It is order expiry date when validity set to GTD (Good Till Da
CancelRejectReason	RejectReason	It is reason if order rejected or canceled
OrderUniqueIdentifier	OrderUniqueIdentifier	Echo back to identify order (20 char length)
OrderLegStatus	OrderLegStatus	It represents type of order i.e. OrderLegStatus
BoLegDetails	BoLegDetails	Bracket Order Leg details i.e. BoLegDetails
IsSpread	IsSpread	Indicates Spread order or normal order
BoEntryOrderID	AppOrderID	Unique order ID
MessageCode	MessageCode	API MessageCode
MessageVersion	MessageVersion	API Message Version
TokenID	TokenID	API TokenID
ApplicationType	ApplicationType	API ApplicationType
SequenceNumber	SequenceNumber	API SequenceNumber

Response Body JSON


```
{
  "type": "success",
  "code": "s-orders-0001",
  "description": "Success trade book",
  "result": [
    {
      "LoginID": "SYMP",
      "ClientID": "SYMP",
      "AppOrderID": 648468731,
      "OrderReferenceID": ""
```


Copy Show Full

Code Examples

SYMPHONY v2.0

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/trades' \
--header 'Authorization: xxxxxx'
```

Copy Show Full

Order History (GET)

Order History provides the complete trail of a particular order, showing all its state changes.
For example: **New → New, New → Partially Filled, Partially Filled → Partially Filled, and Partially Filled → Filled, etc.**

URL

GET `https://developers.symphonyfintech.in/orders?appOrderID=1210999833` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Descriptic
appOrderID	UserID	**N**	Unique orc

Response Body Parameters

Parameter Name	Type	Description
LoginID	UserID	User login ID
ClientID	ClientID	User specific identification
AppOrderID	AppOrderID	Unique order ID
OrderReferenceID	OrderReferenceID	Unique OrderReferenceID to identify multi-leg
GeneratedBy	GeneratedBy	Represents source from which operation was p
ExchangeOrderID	ExchangeOrderID	Unique Order ID generated by exchange
OrderCategoryType	OrderCategoryType	Represents order market type
ExchangeSegment	ExchangeSegment	Exchange Segment
ExchangeInstrumentID	ExchangeInstrumentID	Exchange scrip code or symbol token
OrderSide	OrderSide	Order side (Buy / Sell)
OrderType	OrderType	Type of order (Limit / Market / SL / SL-M)

SYMPHONY v2.0

OrderStatus	OrderStatus	Status of the order
TokenID	TokenID	API Token ID
ApplicationType	ApplicationType	API application type
SequenceNumber	SequenceNumber	API sequence number

Response Body JSON


```
{
  "type": "success",
  "code": "s-orders-0001",
  "description": "Success order history",
  "result": [
    {
      "LoginID": "SYMP1",
      "ClientID": "SYMP1",
      "AppOrderID": 648468730,
      "OrderReferenceID": ""
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders?appOrderID=3727296468' \
--header 'Authorization: xxxxxx' \
--data ''
```

Copy Show Full

Spread Orders

Place Spread Order (POST)

Places a spread order involving spread instruments (buy/sell) in a single request, ensuring all legs are executed together.

URL

POST https://developers.symphonyfintech.in//interactive/orders/spread Copy

SYMPHONY v2.0

Parameter Name	Type	Mandatory	Description
exchangeSegment	ExchangeSegment	Y	Exchange Segment
exchangeInstrumentID	ExchangeInstrumentID	Y	Exchange Scrip code or Symbol Token is uniqu
productType	ProductType	Y	ProductType
action	Action	Y	OrderSide
orderType	OrderType	Y	OrderType
orderDuration	OrderDuration	Y	TimeInForce
quantity	OrderQuantity	Y	Quantity to transact. In terms of Lots
spreadPrice	Price	Y	Spread Price difference
totalPrice	Price	N	Total price used in some spread structures
leg1ExchangeSegment	ExchangeSegment	Y	Exchange segment for Leg 1
leg1ExchangeInstrumentID	ExchangeInstrumentID	Y	Instrument token for Leg 1
leg2ExchangeSegment	ExchangeSegment	Y	Exchange segment for Leg 2
leg2ExchangeInstrumentID	ExchangeInstrumentID	Y	Instrument token for Leg 2
spreadExchangeInstrumentID	ExchangeInstrumentID	Y	Main spread instrument ID
apiOrderSource	ApiOrderSource	N	API Order Source can be third party applicatio give to your order, which will be used to track source.
clientID	ClientID	N	Client ID Mandatory in case of Dealer
userID	UserID	N	User ID authenticated for API

Request Body JSON

```
{
  "exchangeSegment": "NSEFO",
  "exchangeInstrumentID": 13620424,
  "productType": "NRML",
  "action": "BUY",
  "orderType": "LIMIT",
  "orderDuration": "DAY",
```

SYMPHONY v2.0

Copy Show Full

Response Body JSON

```
{
  "type": "success",
  "code": "s-spreadorder-0001",
  "description": "Spread order place successfully",
  "result": {}
}
```

Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/spread' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "exchangeSegment": "NSEFO",
  "exchangeInstrumentID": "39543",
  "productType": "NRML",
  "action": "Buy",
  "orderDuration": "DAY",
  "quantity": "75"
```

Copy Show Full

SpreadOrder Modify (PUT)

Modifies an existing open spread order by updating allowed parameters such as price or quantity for one or more legs.

URL

PUT https://developers.symphonyfintech.in//interactive/orders/spread Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
exchangeOrderID	ExchangeOrderID	**Y**	It is unique OrderID generated by ex

SYMPHONY v2.0

action	Action	**Y**	BUY/SELL instruction
orderDuration	OrderDuration	**Y**	Duration for which the order remair
quantity	OrderQuantity	**Y**	Quantity to transact. In terms of Lo
spreadPrice	Price	**Y**	Spread price for the order
orderID	OrderID	**Y**	Unique order ID generated by RMS
leg1ExchangeSegment	ExchangeSegment	**Y**	Exchange segment of first leg (All I
leg1ExchangeInstrumentID	ExchangeInstrumentID	**Y**	Instrument token of first leg
leg2ExchangeSegment	ExchangeSegment	**Y**	Exchange segment of second leg
leg2ExchangeInstrumentID	ExchangeInstrumentID	**Y**	Instrument token of second leg
spreadExchangeInstrumentID	ExchangeInstrumentID	**Y**	Combined spread instrument toker
apiOrderSource	ApiOrderSource	**N**	Source of order placement (e.g., WI
clientID	ClientID	**N**	Unique client identification

Request Body JSON


```
{
  "exchangeOrderID": "X_174305899",
  "productType": "NRML",
  "action": "BUY",
  "orderDuration": "DAY",
  "quantity": "75",
  "spreadPrice": 900,
  "orderID": "1240992685",
  "leg1ExchangeSegment": "NSEFO",
  "leg1ExchangeInstrumentID": 53216
```


Copy Show Full

Response Body JSON


```
{
  "type": "success",
  "code": "s-updatespreadorder-0001",
  "description": "Success modify spread",
  "result": {}
}
```

SYMPHONY v2.0

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location --request PUT 'https://developers.symphonyfintech.in/1interactive/orders/spread'
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "exchangeOrderID": "124270000004574",
  "productType": "NRML",
  "action": "Buy",
  "orderDuration": "DAY",
  "quantity": "75",
  "spreadPrice": 100
```

Copy | Show Full

SpreadOrder Cancel (DELETE)

Cancels an existing open spread order along with all its associated legs, provided the order has not been fully executed.

URL

DELETE `https://developers.symphonyfintech.in/1interactive/orders/spread?spreadExchangeSegment=NSEFO&spreadExchangeInstrumentID=13687399&exchangeOrderID=X_174305899&orderID=1240992685&apiOrderSource=WebAPI` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
spreadExchangeSegment	ExchangeSegment	**Y**	Exchange segment of the spread ord
spreadExchangeInstrumentID	ExchangeInstrumentID	**Y**	Spread instrument token used for qu
exchangeOrderID	ExchangeOrderID	**N**	Exchange-generated order identifier
orderID	OrderID	**N**	Unique internal order ID assigned by
apiOrderSource	ApiOrderSource	**N**	Source from where the API request o

Response Body Parameters

Parameter Name	Type	Description

SYMPHONY v2.0

Response Body JSON


```
{
  "type": "success",
  "code": "s-deletespreadorder-0001",
  "description": "Spread Order Delete Successfully",
  "result": {
    "OrderID": "800125184000005256"
  }
}
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location --request DELETE 'https://developers.symphonyfintech.in/1interactive/orders/spread?
--header 'Authorization: xxxxxx' \
--data ''
```

Copy Show Full

Spread Order Book (GET)

Retrieves the current status of all spread orders placed by the user, including leg-wise details and execution status.

URL

GET `https://developers.symphonyfintech.in/1interactive/orders/spread?clientID=SYMP&userID=SYMPTEST` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer
userID	userID	**N**	User ID Mandatory

Response Body Parameters

SYMPHONY v2.0

cancelable	Boolean	N	Whether the order can
editable	Boolean	N	Whether the order can
expiration	Expiration	N	Order validity (e.g., DAY
orderID	OrderID	N	Exchange-generated c
orderNumber	OrderNumber	N	System-generated uni
orderType	OrderType	N	OrderType
productType	ProductType	N	ProductType
receivedTime	TimeStamp	N	Time order was receive
status	Status	N	Current order status
exchange	Exchange	N	Exchange segment (e.g
nowOrderStatus	NowOrderStatus	N	Latest updated status
symbolName	SymbolName	N	Symbol of the instrume
spreadExchangeInstrumentID	ExchangeInstrumentID	N	Spread Instrument Tok
participantCode	ParticipantCode	N	Participant Code (if app
rejectReason	Reason	N	Reason for rejection (if

Response Body JSON


```
{
  "orders": [
    {
      "action": "BUY",
      "assetType": "Futures",
      "dTradingSym": "NIFTY",
      "description": "NIFTY 31JUL2025",
      "exchange": "NSEFO",
      "executionPrice": "",
      "filledQuantity": "0"
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

SYMPHONY v2.0

Copy Show Full

GTT Orders

Place GTT Order (POST)

Places a Good Till Triggered (GTT) order that remains active until a specified trigger condition is met. Once the trigger price is reached, the system automatically places the corresponding market or limit order on the exchange.

URL

POST https://developers.symphonyfintech.in/1interactive/orders/gttorder Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**Y**	User-specific identification
source	ApiOrderSource	**Y**	API/Platform source of the request
stopPrice	Price	**Y**	Stop-loss trigger price
limitPrice	Price	**Y**	Limit price at which the order should exe
orderQuantity	OrderQuantity	**Y**	Quantity to transact (in units/lots depe
orderSide	OrderSide	**Y**	BUY or SELL direction of the order
exchangeSegment	ExchangeSegment	**Y**	Exchange segment where the order is p
exchangeInstrumentID	ExchangeInstrumentID	**Y**	Unique token/identifier for the trading i

Request Body JSON

```
{
        "clientID": "SYMP",
        "source": "Twsapi",
        "stopPrice": 1300,
        "limitPrice": 1320,
        "orderQuantity": 75,
        "orderSide": "BUY",
```

SYMPHONY v2.0

Copy Show Full

Response Body JSON

```
{
  "type": "success",
  "code": "s-gttOrderRequest-0001",
  "description": "GTT order request send."
}
```

Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/gttorder' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": 2885,
  "orderSide": "BUY",
  "stopPrice": 1300,
  "limitPrice": 1320,
  "orderQuantity": 75
```

Copy Show Full

GTT Order Modify (PUT)

Allows modification of an Open existing GTT order by updating trigger price or order parameters

URL

PUT https://developers.symphonyfintech.in/1interactive/orders/gttorder Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**Y**	Unique client identification co
orderSide	OrderSide	**Y**	BUY/SELL instruction for the o

SYMPHONY v2.0

exchangeInstrumentID	ExchangeInstrumentID	**Y**	Unique exchange token / scrip
exchangeSegment	ExchangeSegment	**Y**	Exchange segment (e.g., NSEC
appOrderID	AppOrderID	**Y**	Unique RMS-generated order I
modifiedLimitPrice	Price	**N**	New modified limit price
orderCategoryType	OrderCategoryType	**Y**	Order category (NORMAL, STC
modifiedOrderType	OrderType	**N**	Modified order type (Market, L
modifiedProductType	ProductType	**N**	Modified product type (e.g., NI
modifiedOrderQuantity	OrderQuantity	**N**	Modified quantity for the orde
source	Source	**N**	Source of the request (e.g., TV
modifiedStopPrice	Price	**N**	Modified stop-loss trigger pric
participationCode	ParticipationCode	**N**	Order participation code (NON

Request Body JSON


```
{
  "clientID": "SYMP",
  "orderSide": "BUY",
  "orderSessionType": "GTT",
  "exchangeInstrumentID": 2885,
  "exchangeSegment": "NSECM",
  "appOrderID": "1343000794",
  "modifiedLimitPrice": 1360,
  "orderCategoryType": "NORMAL",
  "modifiedOrderType": "Limit"
```


Copy Show Full

Response Body JSON


```
{
  "type": "success",
  "code": "s-modifygttOrder-0001",
  "description": "Modify GTT order request send."
}
```


Copy Show Full

SYMPHONY v2.0

```
curl --location --request PUT 'https://developers.symphonyfintech.in/1interactive/orders/gttorder
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
    "clientID": "SYMP",
    "orderSide": "BUY",
    "orderSessionType": "GTT",
    "exchangeInstrumentID": 2885,
    "exchangeSegment": "NSECM",
    "appOrderID": "1343000794"
```

Copy Show Full

GTT Order Cancel (DELETE)

Cancels an active GTT order, preventing it from being triggered even if the trigger condition is met in the future.

URL

DELETE `https://developers.symphonyfintech.in//interactive/orders/gttorder?clientID=SYMP&appOrderID=1343000794&exchangeSegment=NSECM&exchangeInstrumentID=2885` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**Y**	Unique client identification code
appOrderID	AppOrderID	**Y**	Unique order ID generated by RMS
exchangeSegment	ExchangeSegment	**Y**	Exchange segment where order is pla
exchangeInstrumentID	ExchangeInstrumentID	**Y**	Unique token/ID for the trading instru

Request Body JSON

```
{
  "type": "success",
  "code": "s-cancelgttOrder-0001",
  "description": "Cancel GTT order request send."
}
```

Copy Show Full

SYMPHONY v2.0

```
curl --location --request DELETE 'https://developers.symphonyfintech.in/1interactive/orders/gttorde
--header 'Authorization: xxxxxx'
```

Copy Show Full

GTT OrderBook (POST)

Fetches the list of all GTT orders placed by the user, along with their trigger conditions, order details, and current status.

URL

POST `https://developers.symphonyfintech.in/1interactive/orders/gttorderbook?clientID=SYMP` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**Y**	Description Client ID Mandatory

Response Body Parameters

Parameter Name	Type	Description
LoginID	LoginID	User login ID
ClientID	ClientID	Client code associated with the order
AppOrderID	AppOrderID	Unique application order ID
OrderReferenceID	OrderReferenceID	Reference ID sent during order placement
GeneratedBy	GeneratedBy	Source that generated the order (TWSAPI,
ExchangeOrderID	ExchangeOrderID	Exchange-generated order ID (if available)
OrderCategoryType	OrderCategoryType	NORMAL / SYSTEM / OCO / GTT etc.
ExchangeSegment	ExchangeSegment	Exchange segment (e.g., NSECM)
ExchangeInstrumentID	ExchangeInstrumentID	Token ID of the trading instrument
OrderSide	OrderSide	BUY or SELL

SYMPHONY v2.0

ProductType	ProductType	CNC / MIS / NRML
TimeInForce	TimeInForce	DAY / IOC / None
OrderPrice	Price	Price entered for the order
OrderQuantity	Quantity	Total order quantity
OrderStopPrice	StopPrice	Stop-loss trigger price
OrderStatus	OrderStatus	Current status: New / Filled / Cancelled / R
OrderAverageTradedPrice	Price	Average price at which quantity is traded
LeavesQuantity	LeavesQuantity	Remaining quantity not yet executed
CumulativeQuantity	CumulativeQuantity	Total executed quantity
OrderDisclosedQuantity	DisclosedQuantity	Quantity disclosed to the market
OrderGeneratedDateTime	DateTime	Order generation timestamp
ExchangeTransactTime	DateTime	Exchange transaction timestamp
TradingSymbol	TradingSymbol	Symbol name (e.g., RELIANCE)
LastUpdateDateTime	DateTime	Last updated timestamp
OrderExpiryDate	DateTime	Order validity expiry date
CancelRejectReason	Reason	Reason for cancellation or rejection
OrderUniqueIdentifier	OrderUniqueIdentifier	Unique identifier sent in request
OrderLegStatus	OrderLegStatus	SingleOrderLeg / SpreadFirstLeg / SpreadS
BoLegDetails	BoLegDetails	BO leg details (if applicable)
IsSpread	Boolean	Whether the order is a spread order
BoEntryOrderId	BoEntryOrderId	BO entry order reference
ApiOrderSource	ApiOrderSource	API source (WEB / TWSAPI etc.)
MessageCode	MessageCode	System message code for order update
MessageVersion	MessageVersion	Version of the message format
TokenID	TokenID	Internal token ID

SYMPHONY v2.0

SequenceNumber	SequenceNumber	Order sequence number

Response Body JSON


```
{
  "LoginID": "SYMP",
  "ClientID": "SYMP",
  "AppOrderID": 1343000794,
  "OrderReferenceID": "",
  "GeneratedBy": "TWSAPI",
  "ExchangeOrderID": "",
  "OrderCategoryType": "NORMAL",
  "ExchangeSegment": "NSECM",
  "ExchangeInstrumentID": 2885
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/gttorderbook?clientID=SY
--header 'Authorization: XXXXXX'
```

Copy Show Full

Portfolio (POST)

A user's portfolio consists of long-term equity holdings and short-term positions.

Holding (GET)

The Holdings API allows users to check their long-term holdings with the broker. An authorization token must be included with the request.

URL

GET `https://developers.symphonyfintech.in/1interactive/portfolio/holdings?clientID=SYMP1` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description

SYMPHONY v2.0

Response Body Parameters

Parameter Name	Type	Description
ClientId	UserID	User specific identification
RMSHoldingList	RMSHoldingList	
Holdings	Holdings	
ISIN	ISIN	The standard ISIN representing stocks listed on multiple excha
RMSHoldingId	RMSHoldingId	Unique holding ID
ExchangeNSEInstrumentId	ExchangeInstrumentID	Exchange Scrip code or Symbol Token is unique identifier
ExchangeBSEInstrumentId	ExchangeInstrumentID	Exchange Scrip code or Symbol Token is unique identifier
ExchangeMSEInstrumentId	ExchangeInstrumentID	Exchange Scrip code or Symbol Token is unique identifier
HoldingType	HoldingType	It represents Holding Type i.e. HoldingTypes
HoldingQuantity	Quantity	Holding quantity
CollateralValuationType	ValuationType	ValuationType
Haircut	Haircut	**Haircut** refers to the **percentage reduction applied to the m** when calculating its collateral value
CollateralQuantity	Quantity	Quantities placed as collateral with broker
CreatedBy	CreatedBy	User or system identifier that created the holding record
LastUpdatedBy	LastUpdatedBy	User or system identifier that last modified or updated the hol
CreatedOn	CreatedOn	Date and time when the holding record was first created
LastUpdatedOn	LastUpdatedOn	The date and time when the holding record was last modified c
UsedQuantity	Quantity	Quantity sold from the net holding quantity
IsCollateralHolding	IsCollateralHolding	Indicates whether the holding is being used as **collateral** for m
BuyAvgPrice	Price	Average price at which all shares were bought
IsBuyAvgPriceProvided	IsBuyAvgPriceProvided	Indicates whether the **buy average price** for the holding has b

SYMPHONY v2.0

IsNeedToDelete	IsNeedToDelete	Indicates whether the holding record needs to be **deleted or r** system
CollateralHoldingList	CollateralHoldingList	
Holdings	Holdings	

Response Body JSON


```
{
  "type": "success",
  "code": "s-portfolio-0012",
  "description": "Success holding",
  "result": {
    "ClientId": "SYMP1",
    "RMSHoldingList": [
      {
        "Holdings": {
          "INE457M01133": {
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/portfolio/holdings' \
--header 'Authorization: xxxxxx'
```

Copy Show Full

Position (GET)

The user's portfolio consists of short-term open positions in derivatives (futures and options contracts) and intraday equities.

- Positions in a portfolio remain open until they are sold or until expiry.
- Derivative contracts usually expire in a **monthly sliding window** or a **weekly sliding window** of three periods, called **current, near, and far.**
- Long equity positions are moved to the **holding's portfolio** after the settlement day.

The **Positions API** returns two sets of positions:

- **Net** → The actual, current net position portfolio.
- **Day** → A snapshot of buying and selling activity for that particular day.

SYMPHONY v2.0

URL

GET `https://developers.symphonyfintech.in/1interactive/portfolio/positions?dayOrNet=DayWise` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
dayOrNet	DayOrNet	Y	**Net** → The actual, current net position portfolio **Day** → A snapshot of buying and selling activity for that par
clientID	ClientID	N	Client ID mandatory in case of Dealer

Response Body Parameters

Parameter Name	Type	Description
ExchangeSegment	ExchangeSegment	ExchangeSegment
ExchangeInstrumentID	ExchangeInstrumentID	Exchange Scrip code or Symbol Token is unique ider
ProductType	ProductType	Is the minimum number of shares or units of a secur exchange.
MarketIot	MarketIot	Multiplying factor for currency F&O
Multiplier	Multiplier	Multiplying factor for currency F&O
BuyAveragePrice	Price	Average price at which all shares were bought
SellAveragePrice	Price	Average price at which quantities were sold
OpenBuyQuantity	Quantity	Total bought quantity
OpenSellQuantity	Quantity	Total sold quantity
Quantity	Quantity	Net outstanding quantity
BuyAmount	Amount	Total buy value of position in rupees
SellAmount	SellAmount	Total sell value of position in rupees
NetAmount	NetAmount	Outstanding position value in rupees
UnrealizedMTM	UnrealizedMTM	It's unrealized profit or loss which has not been book
RealizedMTM	RealizedMTM	It's realized profit or loss which has been booked by

SYMPHONY v2.0

MTM	MTM	Mark to market returns (computed based on last clo
BEP	BEP	It's break even point of position
SumOfTradedQuantityAndPriceBuy	SumOfQuantityAndPrice	Total buy value of position in rupees
SumOfTradedQuantityAndPriceSell	SumOfQuantityAndPrice	Total sell value of position in rupees
statisticsLevel	StatisticsLevel	StatisticsLevel
isInterOpPosition	IsInterOpPosition	Interoperability is enabled or disabled
MessageCode	MessageCode	API MessageCode
MessageVersion	MessageVersion	API Message Version
TokenID	TokenID	API TokenID
ApplicationType	ApplicationType	API ApplicationType
SequenceNumber	SequenceNumber	API SequenceNumber

Response Body JSON


```
{
  "type": "success",
  "code": "s-portfolio-0001",
  "description": "Success position",
  "result": [
    {
      "AccountID": "SYMP1",
      "TradingSymbol": "ACC",
      "ExchangeSegment": "NSECM",
      "ExchangeInstrumentID": 22
```


Copy Show Full

Code Examples

CURL Python Go Node-js C# Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/portfolio/positions?dayOrNet=da
--header 'Authorization: xxxxxx'
```

SYMPHONY v2.0

Position Convert (PUT)

Convert Position API enables users to convert their open positions from NRML (Normal) to MIS (Margin Intraday Square-off) or vice versa, provided there is sufficient margin or funds in the account to complete the conversion.

URL

PUT `https://developers.symphonyfintech.in/1interactive/portfolio/positions/convert` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
exchangeSegment	DayOrNet	Y	**Net** → The actual, current net position portfolio **Day** → A snapshot of buying and selling activity fo
exchangeInstrumentID	ClientID	Y	Client ID Mandatory in case of Dealer
oldProductType	ProductType	Y	ProductType
newProductType	ProductType	Y	ProductType
isDayWise	IsDayWise	Y	IsDayWise position conversion
targetQty	Quantity	Y	Quantity that needs to convert
statisticsLevel	StatisticsLevel	Y	StatisticsLevel
isInterOpPosition	sInterOpPosition	Y	Interoperability is enabled or disabled

Request Body JSON

```
{
  "exchangeSegment": "NSECM",
  "exchangeInstrumentID": 2885,
  "oldProductType": "NRML",
  "newProductType": "MIS",
  "isDayWise": true,
  "targetQty": 0,
  "statisticsLevel": "ParentLevel",
  "isInterOpPosition": true
}
```

Copy Show Full

SYMPHONY v2.0


```
{
  "type": "success",
  "code": "s-portfolio-0005",
  "description": "success position convert"
}
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location --request PUT 'https://developers.symphonyfintech.in/1interactive/portfolio/posit
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
    "exchangeSegment": "NSECM",
    "exchangeInstrumentID": 2885,
    "targetQty": 1,
    "isDayWise": true,
    "oldProductType": "NRML",
    "newProductType": "MIS"
```

Copy Show Full

Margin

The Margin API allows clients to fetch the required margin, available margin, and margin utilization details for a given user or for a specific trade order.

It helps verify whether sufficient funds are available before placing an order.

Regular Order Margin (POST)

Calculates the margin required to place a regular order based on order details such as exchange, instrument, order type, price, and quantity, before actually placing the order.

URL

POST https://developers.symphonyfintech.in/1interactive/orders/margindetails Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer

SYMPHONY v2.0

exchange	ExchangeSegment	Y	ExchangeSegment
exchangeInstrumentId	ExchangeInstrumentID	Y	Exchange Scrip code or Symbol Token is
productType	ProductType	Y	ProductType
orderType	OrderType	Y	OrderType
orderSide	OrderSide	Y	OrderSide
quantity	Quantity	Y	Quantity to transact. In terms of Lots
price	Price	Y	The price to execute the order at
stopPrice	Price	Y	The price at which an order should be tr
userID	UserID	Y	User login ID
orderSessionType	--OrderSessionType	Y	Order Session Type

Request Body JSON

```
{
  "clientID": "SYMP",
  "portfolio": [
    {
      "exchange": 1,
      "exchangeInstrumentId": 2885,
      "productType": "NRML",
      "orderType": "LIMIT",
      "orderSide": "BUY",
      "quantity": 1
```

Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
brokerageDeatils	Object	
IsValid	IsValid	Indicates whether the order is valid based on margin checks. `true` means tl
MarginRequired	MarginRequired	Total margin required (in currency units) to place the order
MarginAvailable	MarginAvailable	Total margin currently available in the user's account

SYMPHONY v2.0

ErrorMessage	ErrorMessage	Error message when `IsValid = false`. Empty string means no error.

Response Body JSON


```
{
  "type": "success",
  "code": "s-calculatemargin-0001",
  "description": "Request sent",
  "result": {
    "brokerageDeatils": {
      "IsValid": true,
      "MarginRequired": 150,
      "MarginAvailable": 9816624.5775,
      "MarginShortfall": 0
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/margindetails' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "clientID": "SYMP",
  "portfolio": [
    {
      "exchange": 1,
      "exchangeInstrumentId": 2885,
      "productType": "NRML"
```

Copy Show Full

Modify Regular Order Margin (POST)

Recalculates the margin requirement for a regular order after modifying parameters like price, quantity, Product type helping users understand the updated margin impact before applying the changes.

URL

POST `https://developers.symphonyfintech.in/1interactive/orders/modifyordermargindetails` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description

SYMPHONY v2.0

orderID	AppOrderID	**Y**	Unique Order ID as received in Order Er
instrumentInformation	Array	**Y**	Array of Instrument Information
exchange	ExchangeSegment	**Y**	Exchange Segment
exchangeInstrumentId	ExchangeInstrumentID	**Y**	Exchange Scrip code or Symbol Token i
productType	ProductType	**Y**	ProductType
orderType	OrderType	**Y**	OrderType
orderSide	OrderSide	**Y**	OrderSide
quantity	Quantity	**Y**	Quantity to transact. In terms of Lots
price	Price	**Y**	Price in Rupees. Up to 4 decimal places
stopPrice	Price	**Y**	Price in Rupees. Up to 4 decimal places
userID	UserID	**Y**	UserID of investor client
orderSessionType	OrderSessionType	**Y**	Represents order validity such as DAY, I

Request Body JSON


```
{
  "clientID": "SYMP",
  "orderID": "1210910568",
  "instrumentInformation": {
    "exchange": 1,
    "exchangeInstrumentId": "2885",
    "orderSide": "BUY",
    "orderSessionType": 1,
    "productType": "NRML",
    "orderType": "LIMIT"
```


Copy Show Full

Response Body JSON


```
{
  "type": "success",
  "code": "s-ModifyOrderMarginQueryRequest-0001",
  "description": "Request sent",
  "result": {
    "brokerageDeatils": {
      "IsValid": true,
```

SYMPHONY v2.0

Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/modifyordermargindetai
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
    "clientID": "SYMP",
    "orderID": "1210910568",
    "instrumentInformation": {
        "exchange": 1,
        "exchangeInstrumentId": "2885",
        "orderSide": "BUY"
```

Copy Collapse

Cover Order Margin (POST)

Calculates the margin required to place a cover order by considering both the main order and the compulsory stop-loss order, providing the total margin requirement upfront.

URL

POST https://developers.symphonyfintech.in/1interactive/orders/comargindetails Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer
exchange	ExchangeSegment	**Y**	Exchange Segment
exchangeInstrumentId	ExchangeInstrumentID	**Y**	Exchange Scrip code or Symbol Token is unique ident
entryOrderType	OrderType	**Y**	OrderType
orderSide	OrderSide	**Y**	OrderSide
quantity	Quantity	**Y**	Quantity to transact. In terms of Lots
entryPrice	Price	**Y**	The price to execute the order at

SYMPHONY v2.0

Request Body JSON


```
{
  "clientID": "SYMP",
  "exchange": 1,
  "exchangeInstrumentId": "2885",
  "orderSide": "BUY",
  "entryOrderType": "LIMIT",
  "quantity": 1,
  "entryPrice": 1200,
  "stopPrice": 1180,
  "requestId": 1
```


Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
brokerageDeatils	Object	Contains brokerage, margin, and validation information.
IsValid	IsValid	Indicates whether the order is valid based on margin checks.
MarginRequired	MarginRequired	Total margin required (in currency units) to place the order.
MarginAvailable	MarginAvailable	Total margin currently available in the user's account.
MarginShortfall	MarginShortfall	Margin deficit amount if available margin is insufficient.
ErrorMessage	ErrorMessage	Error message when the order is invalid. Empty string means no e

Response Body JSON


```
{
  "type": "success",
  "code": "s-calculateCoMargin-0001",
  "description": "Request sent",
  "result": {
    "brokerageDeatils": {
      "IsValid": true,
      "MarginRequired": 1200.94,
      "MarginAvailable": 9816624.5775,
      "MarginShortfall": 0
```


Copy Show Full

SYMPHONY v2.0

```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/comargindetails' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "clientID": "SYMP",
  "exchange": 1,
  "exchangeInstrumentId": "2885",
  "orderSide": "BUY",
  "entryOrderType": "LIMIT",
  "quantity": 1
```

Copy Collapse

Modify Cover Order Margin (POST)

Recalculates the margin requirement for a cover order when order parameters are modified, reflecting any changes in margin due to updated price, quantity, or stop-loss values.

URL

POST `https://developers.symphonyfintech.in/1interactive/orders/comodifymargindetails` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer
exchange	ExchangeSegment	**Y**	Exchange Segment
exchangeInstrumentId	ExchangeInstrumentID	**Y**	Exchange Scrip code or Symbol Token is unique ident
entryOrderType	OrderType	**Y**	OrderType
orderSide	OrderSide	**Y**	OrderSide
quantity	Quantity	**Y**	Quantity to transact. In terms of Lots
entryPrice	Price	**Y**	The price to execute the order at
stopPrice	Price	**Y**	A stoploss price is the price in a stop order that trigge order.
orderID	AppOrderID	**Y**	Unique order ID

SYMPHONY v2.0


```
{
  "clientID": "SYMP",
  "exchange": 1,
  "exchangeInstrumentId": 2885,
  "orderSide": "BUY",
  "entryOrderType": "LIMIT",
  "quantity": 1,
  "entryPrice": 1200,
  "stopPrice": 1180,
  "requestId": 1
```


Copy Show Full

Response Body Parameters

Parameter Name	Type	Description
brokerageDeatils	Object	
IsValid	IsValid	Indicates whether the order is valid based on margin checks. `true` means tl
MarginRequired	MarginRequired	Total margin required (in currency units) to place the order.
MarginAvailable	MarginAvailable	Total margin currently available in the user's account.
MarginShortfall	MarginShortfall	Margin deficit amount if available margin is insufficient.
ErrorMessage	ErrorMessage	Error message when `IsValid = false`. Empty string means no error.

Response Body JSON


```
{
  "type": "success",
  "code": "s-COModifyMarginQueryRequest-0001",
  "description": "Request sent",
  "result": {
    "brokerageDeatils": {
      "IsValid": true,
      "MarginRequired": 1200,
      "MarginAvailable": 9816624.5775,
      "MarginShortfall": 0
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

SYMPHONY v2.0

```
--header 'Content-Type: application/json' \
--data '{
  "clientID": "SYMP",
  "exchange": 1,
  "exchangeInstrumentId": 2885,
  "orderSide": "BUY",
  "entryOrderType": "LIMIT",
  "quantity": 1
```

Copy Collapse

Bracket Order Margin (POST)

Calculates the margin required to place a bracket order by considering the main order along with the associated target and stop-loss orders, giving users clarity on total margin usage before placing the order.

URL

POST `https://developers.symphonyfintech.in/1interactive/orders/bomargindetails` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Description
clientID	ClientID	**N**	Client ID Mandatory in case of Dealer
exchange	ExchangeSegment	**Y**	Exchange Segment
exchangeInstrumentId	ExchangeInstrumentID	**Y**	Exchange Scrip code or Symbol Token i
entryOrderType	OrderType	**Y**	OrderType
orderSide	OrderSide	**Y**	OrderSide
quantity	Quantity	**Y**	Quantity to transact. In terms of Lots
limitPrice	Price	**Y**	Price in Rupees. Up to 4 decimal places
stopLoss	Price	**Y**	Price in Rupees. Up to 4 decimal places
squarOff	SquarOff	**Y**	Represents the profit-taking offset val

Request Body JSON

```
{
  "clientID": "SYMP",
```

SYMPHONY v2.0


```
"orderSide": "BUY",
"entryOrderType": "LIMIT",
"quantity": 1,
"limitPrice": 1200,
"squarOff": 1290,
```


Copy Show Full

Response Body JSON


```
{
  "type": "success",
  "code": "s-calculateBoMargin-0001",
  "description": "Request sent",
  "result": {
    "brokerageDeatils": {
      "IsValid": true,
      "MarginRequired": 1200,
      "MarginAvailable": 9816624.5775,
      "MarginShortfall": 0
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java


```
curl --location 'https://developers.symphonyfintech.in/1interactive/orders/bomargindetails' \
--header 'Authorization: xxxxxx' \
--header 'Content-Type: application/json' \
--data '{
  "clientID": "SYMP",
  "exchange": 1,
  "exchangeInstrumentId": 2885,
  "orderSide": "BUY",
  "entryOrderType": "LIMIT",
  "quantity": 1
```


Copy Collapse

Exchange Status (GET)

Exchange Status indicates the availability of eligible exchanges for normal trading. It allows users to determine whether they can place an order or not.

URL

SYMPHONY v2.0

Request Body Parameters

Parameter Name	Type	Mandatory	Description
userID	UserID	Y	User specific identification

Response Body Parameters

Parameter Name	Type	Description
exchangeSegment	ExchangeSegment	ExchangeSegme
exchangeMarketType	MarketType	MarketType
exchangeTradingSession	TradingSession	TradingSession

Response Body JSON


```
{
  "type": "success",
  "code": "s-status-0002",
  "description": "Exchange status available",
  "result": {
    "marketStatus": [
      {
        "exchangeSegment": "NSECM",
        "exchangeMarketType": "AUCTION",
        "exchangeTradingSession": "NormalEnd"
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

```
curl --location 'https://developers.symphonyfintech.in/1interactive/status/exchange?userID=SYMPTEST
--header 'Authorization: xxxxxx'
```

Copy Show Full

SYMPHONY v2.0

Exchange Message (GET)

The exchange sends various messages to connected brokers, such as information about bans, circuit limits, and news about listed companies. Users can access messages relevant to them using this API.

URL

GET `https://developers.symphonyfintech.in/1interactive/messages/exchange?exchangeSegment=NSECM` Copy

Request Body Parameters

Parameter Name	Type	Mandatory	Descrip
exchangeSegment	ExchangeSegment	Y	Exchan

Response Body Parameters

Parameter Name	Type	Description
exchangeSegment	ExchangeSegment	ExchangeSegment
ExchangeTimeStamp	ExchangeTimeStamp	Date and time provided by the exchange when a specific event or m
BroadcastMessage	BroadcastMessage	Message Send by exchange

Response Body JSON


```
{
  "type": "success",
  "code": "s-messages-0001",
  "description": "Exchange messages",
  "result": {
    "messageList": [
      {
        "ExchangeSegment": 2,
        "ExchangeTimeStamp": 1452777915,
        "BroadcastMessage": "Listing -> CORPORATE ANNOUNCEMENTS"
```


Copy Show Full

Code Examples

CURL | Python | Go | Node-js | C# | Java

SYMPHONY v2.0

```
--data ''
```

Copy Show Full

Socket Streaming

Introduction

order to get streaming interactive events, you need to use the socket.io library.
For more information, you can visit: https://socket.io/.

Real-Time Streaming Flow

Once the connection is established, real-time streaming can be achieved by listening to the socket for relevant interactive events. You can receive the corresponding streaming data on these events.

Follow the steps below to enable real-time streaming of interactive events:

- **Establish the socket connection.**
- **Register interactive events on the socket connection.**
- **Listen for real-time interactive events to receive live updates.**

Establishment of socket connection

```
import socketio

url = "https://developers.symphonyfintech.in"    # base url
userID = "SYMP"         #userID received from login api
token = "xxxxxx"         #Token generated from login api
apiType = "INTERACTIVE"
socketio_path = "/1interactive/socket.io"       #note:- /1interactive is endpoint you have reciev
logger = False  # or "True"

sio = socketio.Client()
```

Copy Show Full

Note:

- XTS uses Socket.io protocol for the Interactive order related data.
- Users must note that some programming languages does not support socket.io protocol directly, so they need to find compatible websocket libraries.

This event triggers when socket connection is successfull.

```
@sio.on("connect")
def connect():
  print("Connection Established")
```

Copy Show Full

Joined

If a successful connection is established with the server, this event will be raised to indicate that the user has been connected.

```
@sio.on("joined")
def on_joined(data):
    print("Socket Joined", data)
```

Copy Show Full

Error

In case of any error with the server, this event will be raised along with the cause of the error. For example, if you send an incorrect token while connecting, the server will raise this event and disconnect your socket connection.

```
@sio.on("error")
def on_error(data):
    print("Error", data)
```

Copy Show Full

Disconnect

In case a socket disconnection occurs with the server, this event will be raised to indicate the disconnection.

SYMPHONY v2.0

```
    print("Disconnected")
```

Copy Show Full

Orders

To listen for order state change events such as New, Filled, Partially Filled, etc., register for this event.

```
@sio.on("ordert")
def on_order(data):
    print("Order Received:- ",data)
```

Copy Show Full

Below is order object you will receive when any of your order state changes

```
{
  "LoginID": "SYMP1",
  "ClientID": "SYMP1",
  "AppOrderID": 648468730,
  "OrderReferenceID": "",
  "GeneratedBy": "WSAPI",
  "ExchangeOrderID": "1005239196374108",
  "OrderCategoryType": "NORMAL",
  "ExchangeSegment": "NSECM",
  "ExchangeInstrumentID": 16921
```

Copy Show Full

Trade

When any order gets executed (fully or partially filled), a new trade event will be generated. The server will raise trade events, and you can listen to these events by registering for them.

```
@sio.on("trade")
def on_trade(data):
    print("Trade Recieved:- ", data)
```

SYMPHONY v2.0

Below is trade object you will receive when any trade happens on your account.


```
{
  "LoginID": "YMS",
  "ClientID": "YMSCLI",
  "AppOrderID": 648468731,
  "OrderReferenceID": "",
  "GeneratedBy": "TWSAPI",
  "ExchangeOrderID": "1005239196374109",
  "OrderCategoryType": "NORMAL",
  "ExchangeSegment": "NSECM",
  "ExchangeInstrumentID": 16921
```


Copy Show Full

Position

If any position change occurs in your account, the server will raise this event to indicate the updated position. By registering for this event, you can track position changes.

```
@sio.on("position")
def on_position(data):
    print("Position Recieved:- ", data)
```

Copy Show Full

Below is Position object that you will receive when any position change happens on your account.


```
{
  "LoginID": "SYMP1",
  "AccountID": "SYMP1",
  "TradingSymbol": "ACC",
  "ExchangeSegment": "NSECM",
  "ExchangeInstrumentID": "22",
  "ProductType": "CNC",
  "Multiplier": 1,
  "Marketlot": 1,
  "BuyAveragePrice": 41.78
```


Copy Show Full

Trade conversion

SYMPHONY v2.0

```
@sio.on("tradeConversion")
def on_tradeconversion(data):
    print("TradeConversion Recieved:- ", data)
```

Copy Show Full

Below is Position object that you will receive when any position change happens on your account.


```
{
  "LoginID": "SYMP1",
  "ClientID": "SYMP1",
  "Success": true,
  "ErrorMessage": "",
  "OriginalProduct": "MIS",
  "TargetProduct": "NRML",
  "OriginalQty": 10,
  "TargetQty": 10,
  "EntityType": "Client"
```


Copy Show Full

GTT Order

When any GTT order gets executed, a new trade event will be generated. The server will raise trade events & you can listen to this event by registering it.

```
@sio.on("gttOrder")
def on_gttOrder(data):
    print("GTT Order Recieved:- ", data)
```

Copy Show Full

Below is the gttOrder object that you will receive when any gttOrder change happens on your account.


```
{
  "LoginID": "SYMP",
  "ClientID": "SYMP1",
  "AppOrderID": "1311100040",
  "OrderReferenceID": "",
  "GeneratedBy": "Web",
  "ExchangeOrderID": "",
  "OrderCategoryType": "NORMAL",
```

SYMPHONY v2.0

GTT Order Rejection

When any GTT order gets rejected, a new trade event will be generated. The server will raise trade events & you can listen to this event by registering it.

```
@sio.on("gttOrderRejection")
def on_gttOrderRejection(data):
    print("GTT Order Rejection Recieved:- ", data)
```

Copy Show Full

Below is gttOrderRejection object that you will receive when any gttOrder rejection change happens on your account.


```
{
  "Type": 783,
  "Data": "",
  "ErrorString": "Gateway:Freeze Quantity: Required Limit[1000000] Freeze Quantity Limit[6]",
  "ClientID": "SYMP",
  "UserID": "SYMP1"
}
```


Copy Show Full

Market Status Update

marketStatusUpdate

```
@sio.on("marketStatusUpdate")
def on_marketStatusUpdate(data):
    print("marketStatusUpdate Recieved:- ", data)
```

Copy Show Full

Below is marketStatusUpdate.

SYMPHONY v2.0

```
  "exchangeSegment": "BSECM",
  "marketSession": "NormalStart"
}
```

Copy Show Full

Exchange Adapter State

exchangeAdapterState

```
@sio.on("exchangeAdapterState")
def on_exchangeAdapterState(data):
    print("exchangeAdapterState Recieved:- ", data)
```

Copy Show Full

Below is exchangeAdapterState.

Copy Show Full

Logout

When your session is logged out by the server, it will raise this event. By registering for this event, you can track your session state.

```
@sio.on("logout")
def on_logout(data):
    print("Logout Recieved:- ", data)
```

Copy Show Full

SYMPHONY v2.0

MTM Calculation

MTM represents the realized and unrealized profit or loss calculated using the user's current position and the instrument's Last Traded Price (LTP). MTM values are not provided in Interactive API responses; users must compute MTM on their end using current position that can be fetched by GET Position api or Position data recieved on Interactive websocket, LastTradedPrice for particular instrument can be fetched using marketdata apis & websocket. **referring** the sample logic provided below.

```
openBuyQty = int(position["OpenBuyQuantity"])
openSellQty = int(position["OpenSellQuantity"])

# Realized MTM
def safe_div(numerator, denominator):
    return float(numerator) / float(denominator) if denominator else 0.0

if openBuyQty == 0 or openSellQty == 0:
    realizedMTM = 0.0
else:
```

Copy Show Full